Descriptive vs. inferential statistics

In this video, you will learn

- The difference between “descriptive statistics” and “inferential statistics.”
- Why it is important to make the distinction clear.
- Why I and many others are really annoyed that the terms are so similar!

ûdemy

Statistical homonym

Descriptive statistics

Describing the characteristics of a dataset:

- Mean, median, mode
- Variance
- Kurtosis, skew
- Distribution shape
- Spectrum

Notes: No relation to population; no generalization to other datasets or groups.

Inferential statistics

Using features of the dataset to make claims about a population.

- P-value
- T/F/chi-square value
- Confidence intervals
- Hypothesis testing

Notes: The entire purpose is to relate data features to populations or generalize to other groups!

ûdemy

Accuracy, precision, resolution

In this video, you will learn

- The terms accuracy, precision, and resolution.
- Why these concepts are important in statistics, measurement, and empirical science more generally.
- Other terms that are sometimes used.

ûdemy

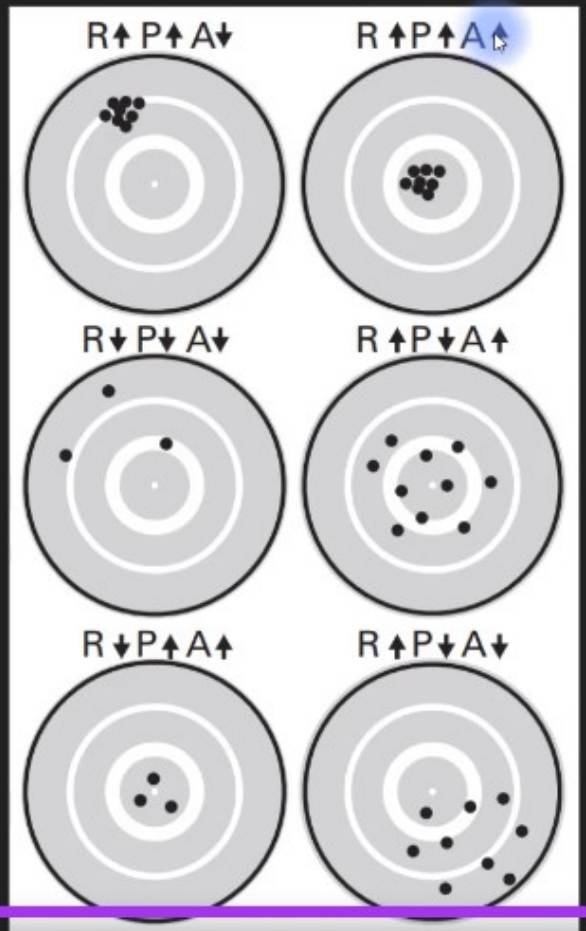
A, P, and R: definitions

Accuracy: The relationship between the measurement and the actual Truth (inversely related to *bias*).

Precision: The certainty of each measurement (inversely related to *variance*).

Resolution: The number of data points per unit measurement (time, space, individual, ...).

A, P, and R: pictures



Taken from *Cohen. ANTS. MIT Press, 2014*

	Actual Positive	Actual Negative
Predicted Positive	True Positive (TP)	False Positive (FP)
Predicted Negative	False Negative (FN)	True Negative (TN)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{Specificity} = \frac{TN}{TN+FP}$$

Durum	Öncelikli Metrik	Hedef
Hatalı "Pozitif" alarmı çok maliyetli ise	Precision	Yanlış alarmları en aza indir
Gerçek bir vakayı "Negatif" sanmak (atlamak) çok maliyetli ise	Recall	Hiçbir gerçek vakayı kaçırma
Her iki hata türü de eşit derecede önemliyse	F1 Skoru	Dengeli bir performans yakala

Key: **tp** = True Positive, **tn** = True Negative, **fp** = False Positive, **fn** = False Negative

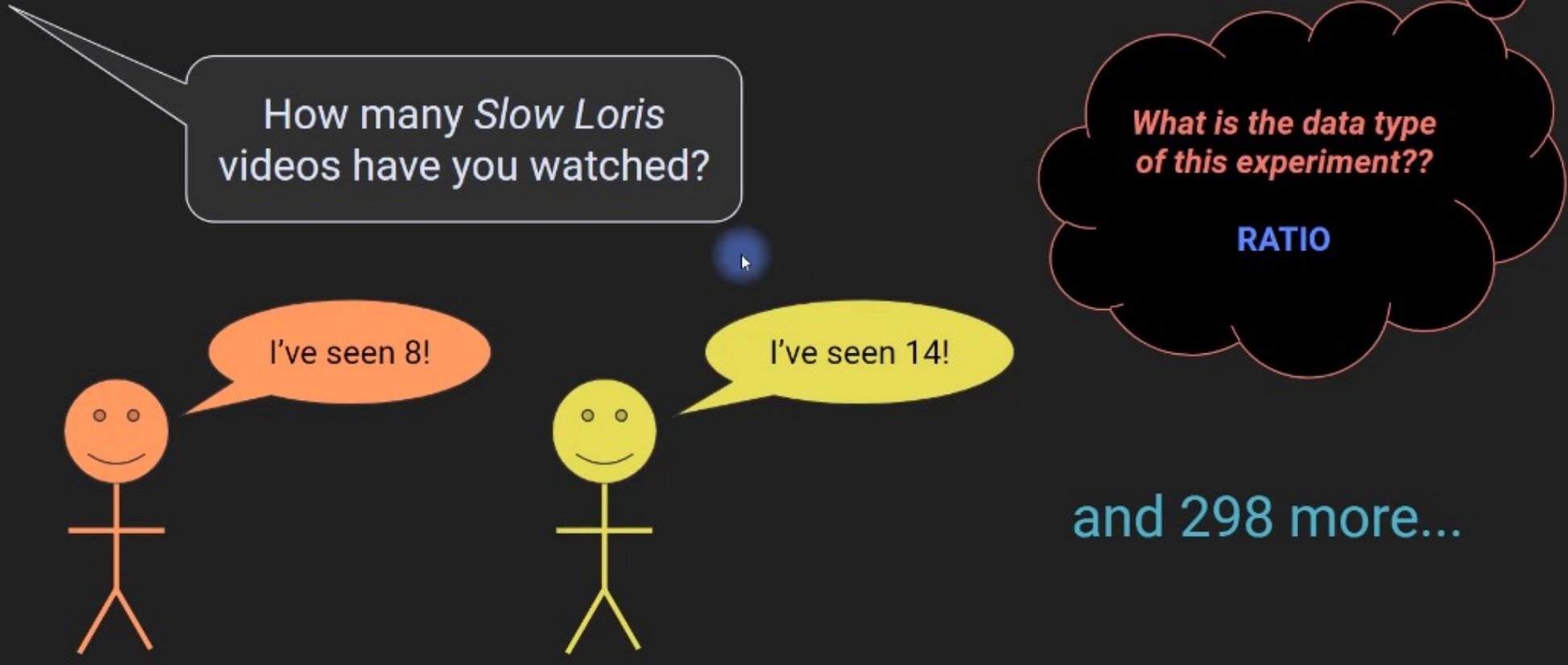
Metric Name	Metric Forumla	Code	When to use
Accuracy	$\textbf{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$	`tf.keras.metrics.Accuracy()` or `sklearn.metrics.accuracy_score()`	Default metric for classification problems. Not the best for imbalanced classes.
Precision	$\textbf{Precision} = \frac{tp}{tp + fp}$	`tf.keras.metrics.Precision()` or `sklearn.metrics.precision_score()`	Higher precision leads to less false positives.
Recall	$\textbf{Recall} = \frac{tp}{tp + fn}$	`tf.keras.metrics.Recall()` or `sklearn.metrics.recall_score()`	Higher recall leads to less false negatives.
F1-score	$\textbf{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	`sklearn.metrics.f1_score()`	Combination of precision and recall, usually a good overall metric for a classification model.
Confusion matrix	NA	`Custom function` or `sklearn.metrics.confusion_matrix()`	When comparing predictions to truth labels to see where model gets confused. Can be hard to use with large numbers of classes.

Data distributions

In this video, you will learn

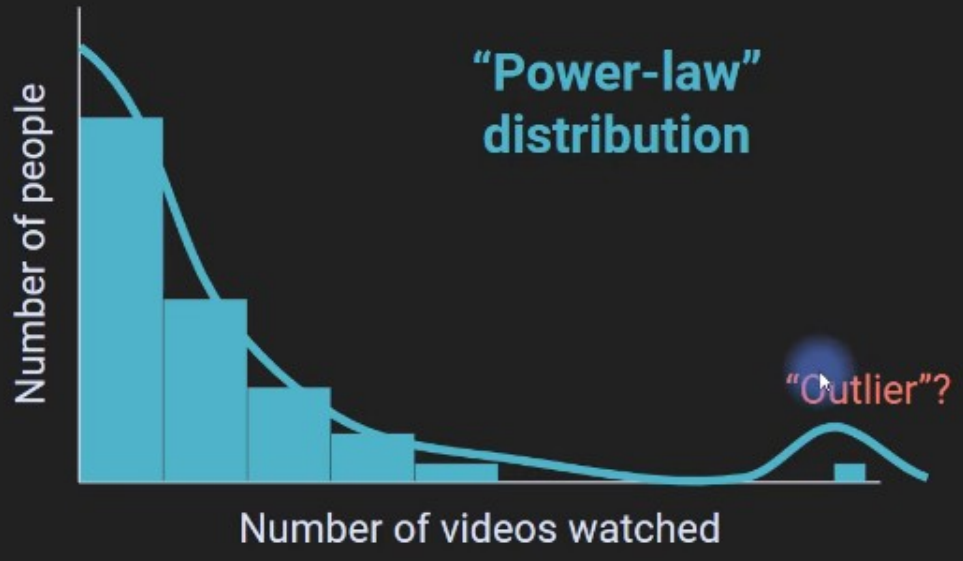
- What a “data distribution” means.
- Examples of different distributions.
- How distributions are used in statistics.
- Segue to quantitative measures of distributions.

Building up a data distribution



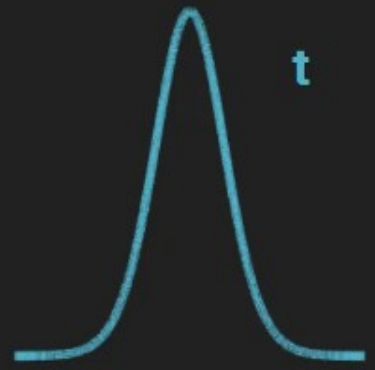
The shape of data distributions

How many *Slow Loris* videos have you watched?



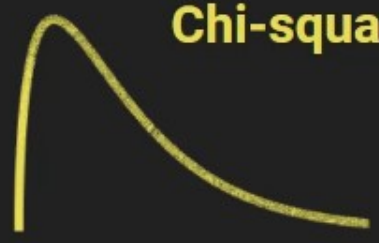
Examples of data distributions

Gaussian
("bell curve")



t

F



Chi-square (χ^2)

Who cares about distributions?

Most statistical procedures are based on assumptions about distributions.

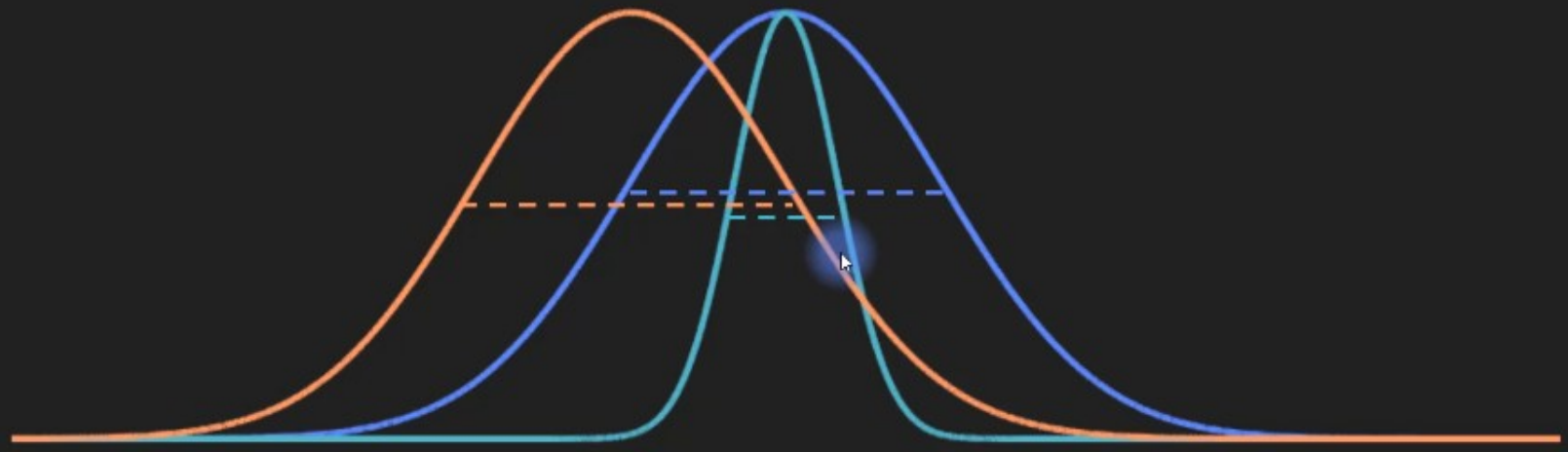
Knowing these distributions is necessary for appropriately applying statistics.

Data distributions provide insights into nature.

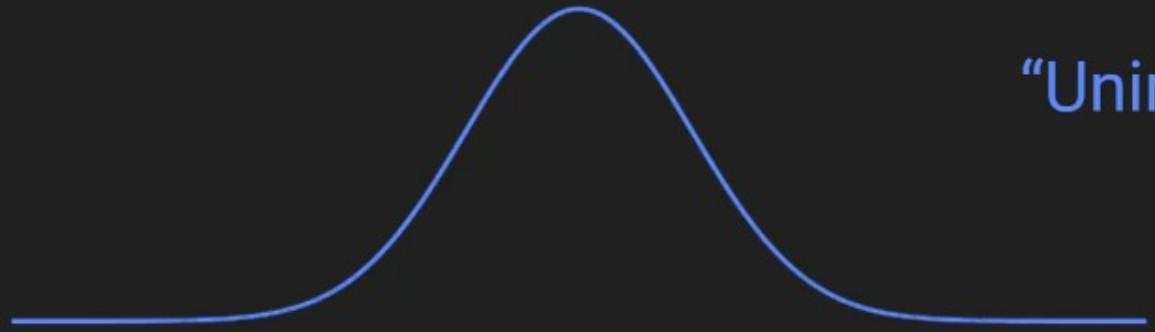
Physical and biological systems are modeled using distributions.

Qualities of distributions

Qualities of distributions



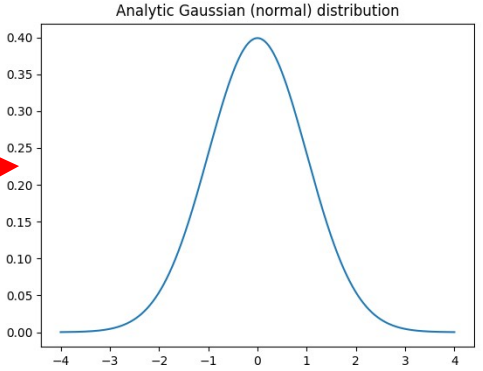
“Modes” of a distribution

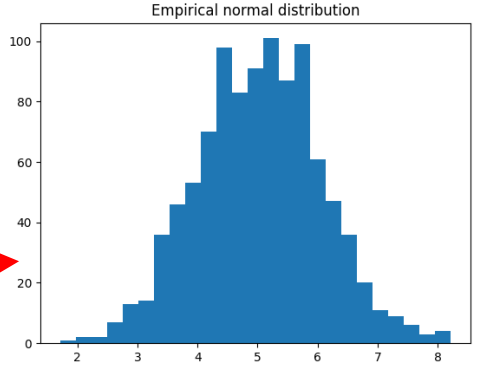


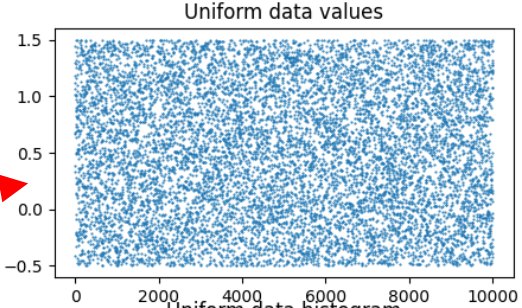
“Unimodal” distribution

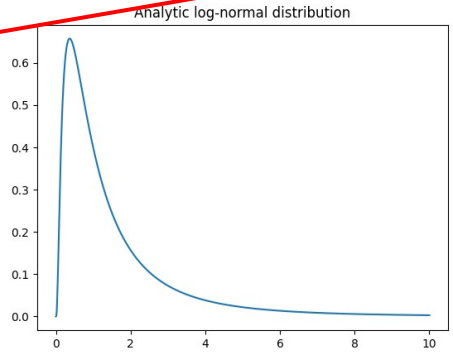
“Bimodal” distribution
“Multimodal” distribution

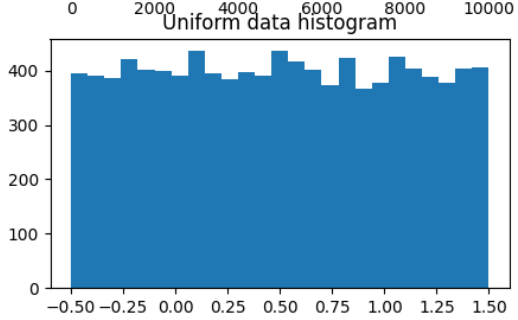
```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
## Gaussian
# number of discretizations
N = 1001
x = np.linspace(-4,4,N)
gausdist = stats.norm.pdf(x)
plt.plot(x,gausdist)
plt.title('Analytic Gaussian (normal) distribution')
plt.show()
# is this a probability distribution?
print(sum(gausdist))
# try scaling by dx...
## Normally-distributed random numbers
# parameters
stretch = 1 # variance (square of standard deviation)
shift   = 5 # mean
n       = 1000
# create data
data = stretch*np.random.randn(n) + shift
# plot data
plt.hist(data,25)
plt.title('Empirical normal distribution')
plt.show()
## Uniformly-distributed numbers
# parameters
stretch = 2 # not the variance
shift   = .5
n       = 10000
# create data
data = stretch*np.random.rand(n) + shift-stretch/2
# plot data
fig,ax = plt.subplots(2,1,figsize=(5,6))

ax[0].plot(data,'.',markersize=1)
ax[0].set_title('Uniform data values')

ax[1].hist(data,25)
ax[1].set_title('Uniform data histogram')

plt.show()

N = 1001
x = np.linspace(0,10,N)
lognormdist = stats.lognorm.pdf(x,1)

plt.plot(x,lognormdist)
plt.title('Analytic log-normal distribution')
plt.show()
```



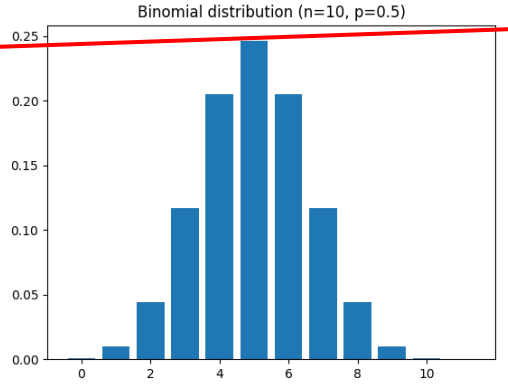


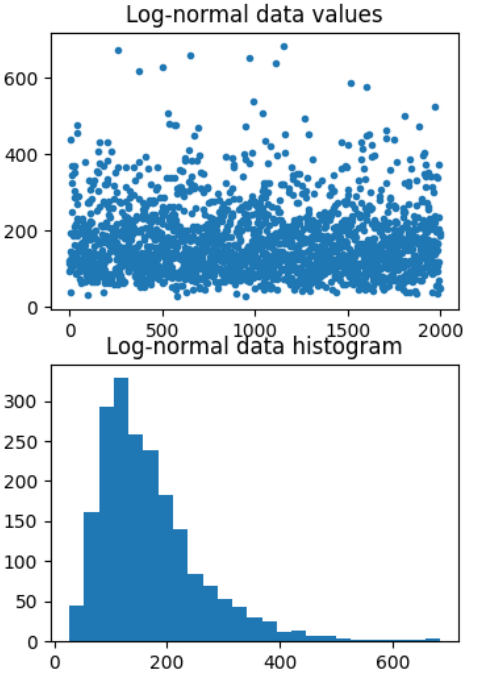


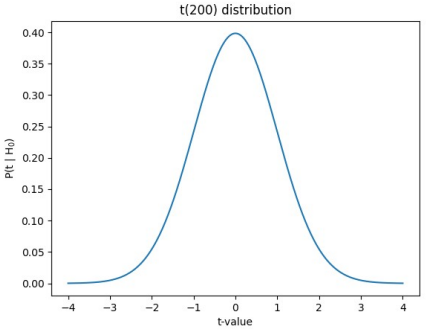


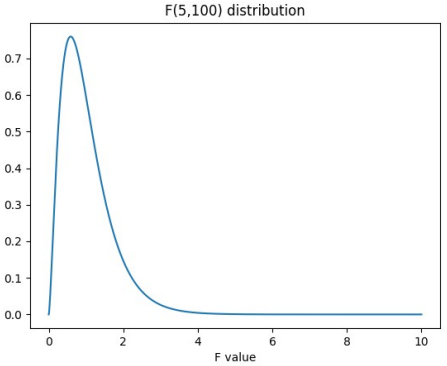


```
## empirical log-normal distribution
shift   = 5  # equal to the mean?
stretch = .5 # equal to standard deviation?
n = 2000     # number of data points
# generate data
data = stretch*np.random.randn(n) + shift
data = np.exp( data )
# plot data
fig,ax = plt.subplots(2,1,figsize=(4,6))
ax[0].plot(data,'.')
ax[0].set_title('Log-normal data values')
ax[1].hist(data,25)
ax[1].set_title('Log-normal data histogram')
plt.show()
## binomial
# a binomial distribution is the probability
# of K heads in N coin tosses,
# given a probability of p heads
#  (e.g., .5 is a fair coin).
n = 10 # number on coin tosses
p = .5 # probability of heads
x = range(n+2)
bindist = stats.binom.pmf(x,n,p)
plt.bar(x,bindist)
plt.title('Binomial distribution (n=%s, p=%g)'%(n,p))
plt.show()

## t
x  = np.linspace(-4,4,1001)
df = 200
t  = stats.t.pdf(x,df)
plt.plot(x,t)
plt.xlabel('t-value')
plt.ylabel('P(t | H$_0$)')
plt.title('t(%g) distribution'%df)
plt.show()

## F
# parameters
num_df = 5   # numerator degrees of freedom
den_df = 100 # denominator df
# values to evaluate
x = np.linspace(0,10,10001)
# the distribution
fdist = stats.f.pdf(x,num_df,den_df)
plt.plot(x,fdist)
plt.title(f'F({num_df},{den_df}) distribution')
plt.xlabel('F value')
plt.show()
```









The beauty and simplicity of Normal

In this video, you will learn

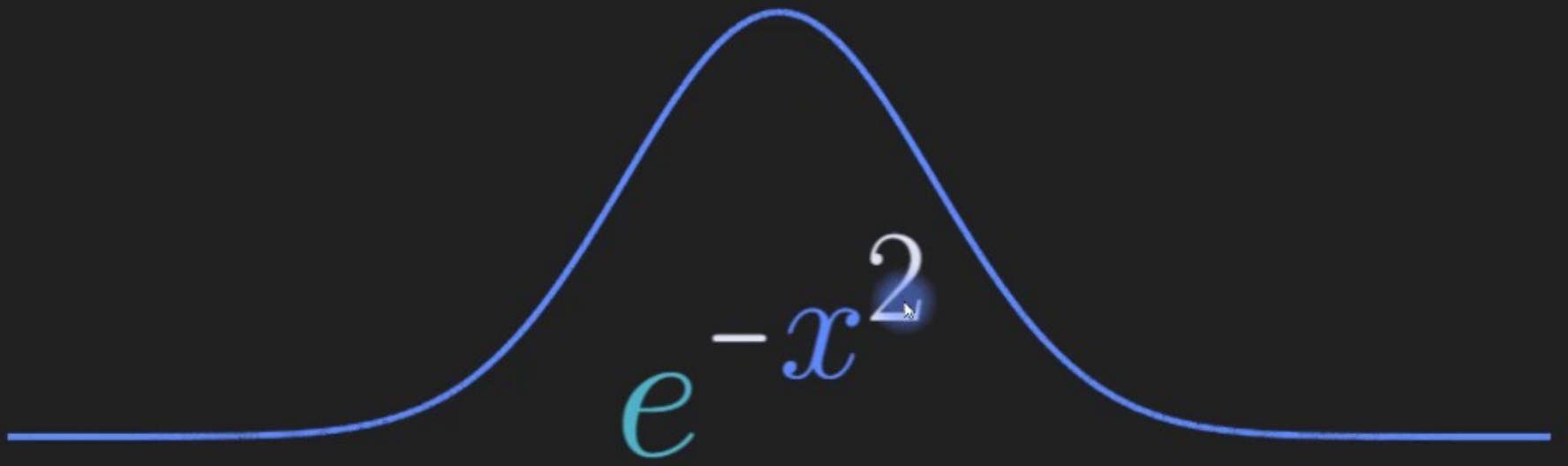
- The breathtaking awesomeness of the normal distribution.

udemy

The complete and the simple

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \quad \Rightarrow \quad e^{-x^2}$$

The normal distribution



Normal / standard / bell curve / Gaussian

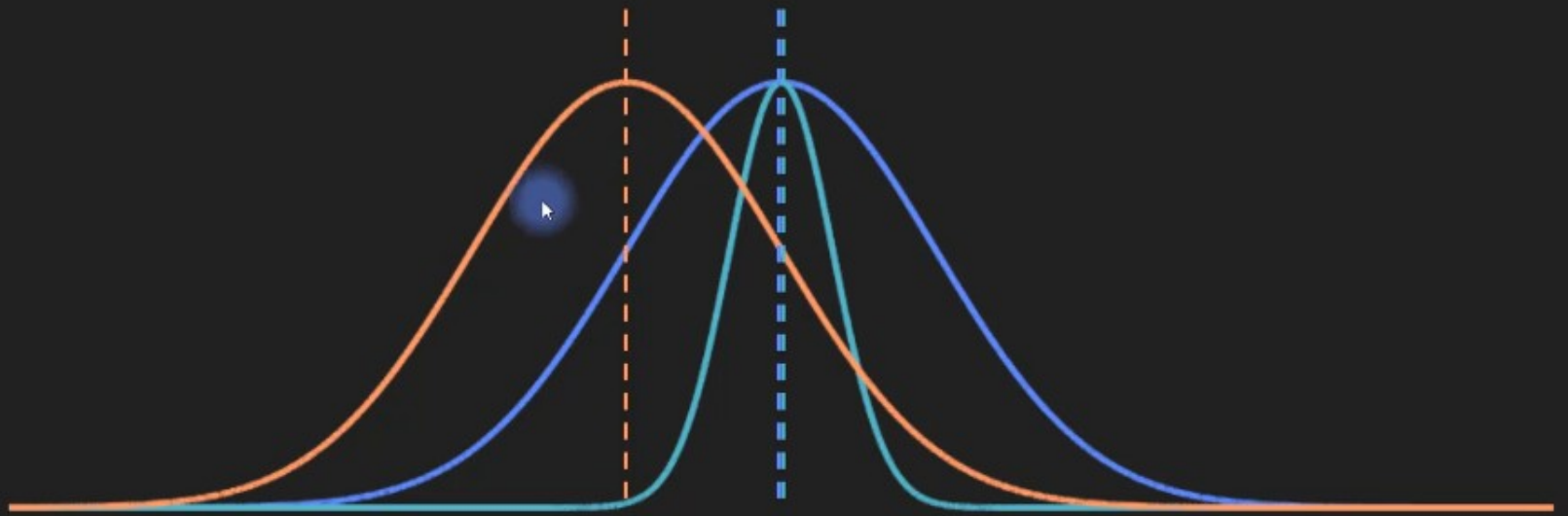
Measures of central tendency

In this video, you will learn

- What the term “central tendency” means.
- The formulas for the three most commonly used measures of central tendency (mean, median, mode).
- Which kinds of data are appropriate for these measures.
- A few real-world examples.

udemy

Central tendency



Note: Central tendency is different from "expected value." Expected value is the data value times its probability of occurrence. More on this in the Probability Theory section!

udemy

Mean (a.k.a. arithmetic mean a.k.a. average)

Formula: $\bar{x} = n^{-1} \sum_{i=1}^{n} x_i$

Notation: $\bar{x} = \mu = \mu_x$

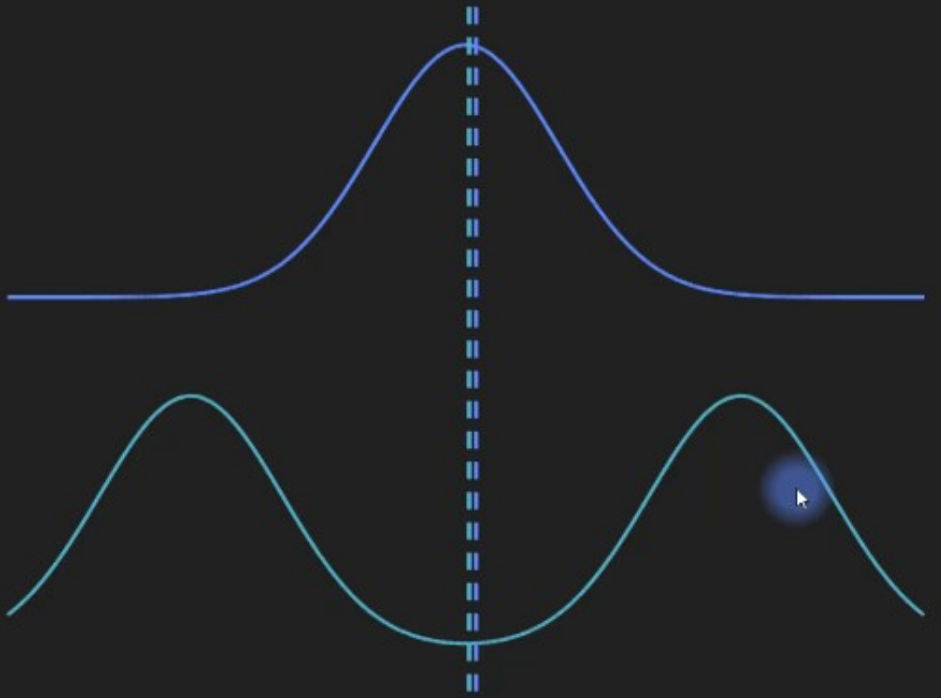
Mean (a.k.a. arithmetic mean a.k.a. average)

Formula: $\bar{x} = n^{-1} \sum_{i=1}^{n} x_i$

Suitable for:
roughly normally distributed data.

Suitable data types:
Interval, ratio

"Failure" scenarios:



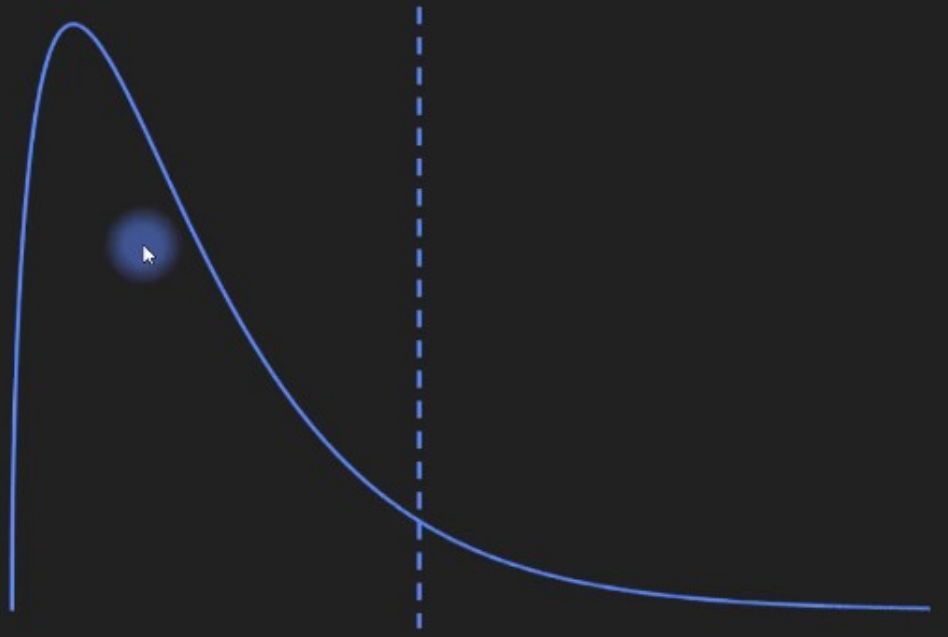
Mean (a.k.a. arithmetic mean a.k.a. average)

Formula: $\bar{x} = n^{-1} \sum_{i=1}^{n} x_i$

Suitable for:
roughly normally distributed data.

Suitable data types:
Interval, ratio

"Failure" scenarios:



Mean for other data types

Is the mean suitable for discrete data?
Example: Average US family has 1.9 children.

Is the mean suitable for ordinal data?
Example: Average course rating of 4.3 stars (out of 1-5 stars; whole-star ratings only).

Is the mean suitable for nominal data?
Example: Average person likes 1.7 ice cream flavor (1=chocolate, 2=vanilla, 3=strawberry).

Mean for other data types

Is the mean suitable for discrete data?

Conclusions:

The mean is best applied to interval and ratio data.

The mean of discrete and ordinal data can be useful, but must be carefully interpreted.

The mean is not appropriate for nominal data.

Median

Formula: $x_i, \quad i = \frac{n+1}{2}$

Example:

x = [0,4,1,-2,7]

med(x) = [-2,0,1,4,7]

Suitable for:
Unimodal distributions.

Suitable data types:
Interval, ratio

Median

Formula: $x_i, \quad i = \frac{n+1}{2}$

Suitable for:
Unimodal distributions.

Suitable data types:
Interval, ratio

Example:

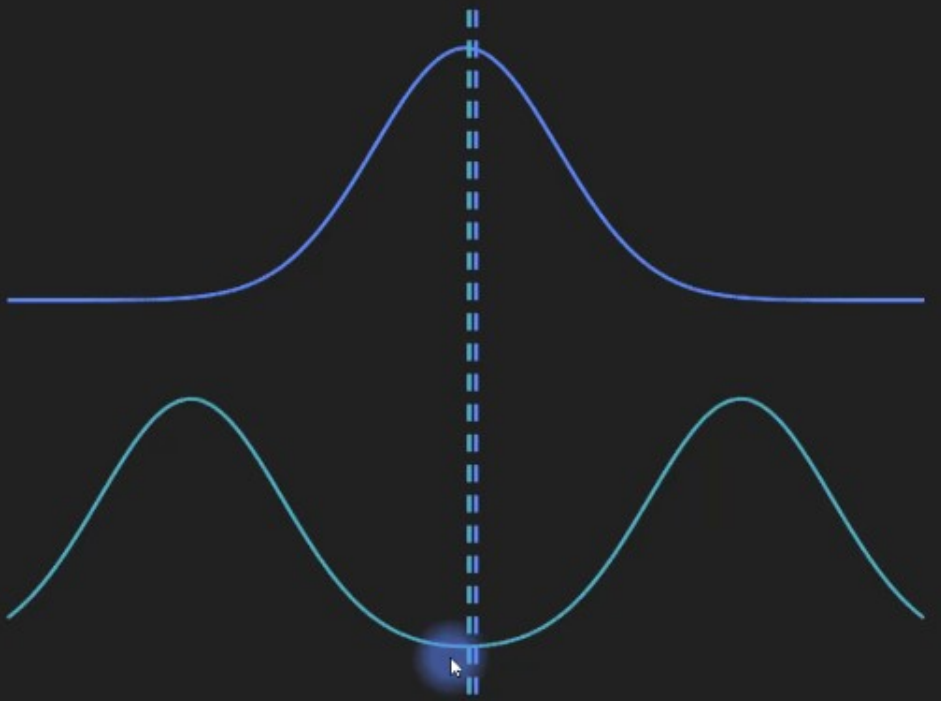
x = [10,0,4,1,-2,7]

med(x) = [-2,0,1,4,7,10]
med(x) = 2.5

Median

Formula: $x_i, \quad i = \frac{n+1}{2}$

"Failure" scenario:



Suitable for:
Unimodal distributions.

Suitable data types:
Interval, ratio

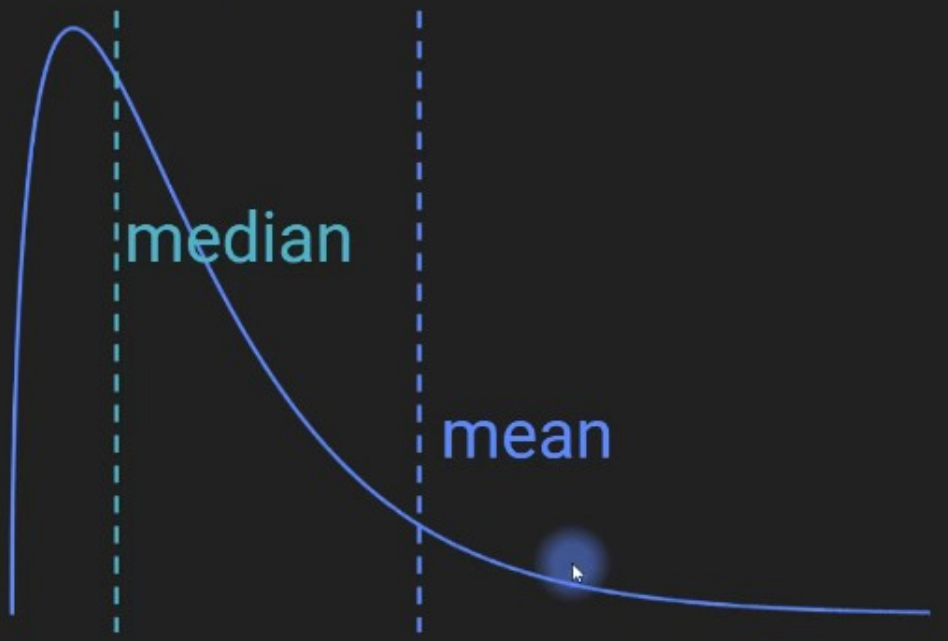
Median

Formula: $x_i, \quad i = \frac{n+1}{2}$

Suitable for:
Unimodal distributions.

Suitable data types:
Interval, ratio

Compared to mean:



udemy

Mode

Formula:
Most common value

Suitable for:
Any distribution.

Suitable data types:
Any (numerical data should be converted to discrete)

Example:

x = [0,0,1,1,1,2,3,4]

mode(x) = 1

udemy

Mode

Formula:
Most common value

Suitable for:
Any distribution.

Suitable data types:
Any (numerical data should be converted to discrete)

Example:

x = [0,0,1,1,1,2,3,0,4]

mode(x) = 0,1

udemy

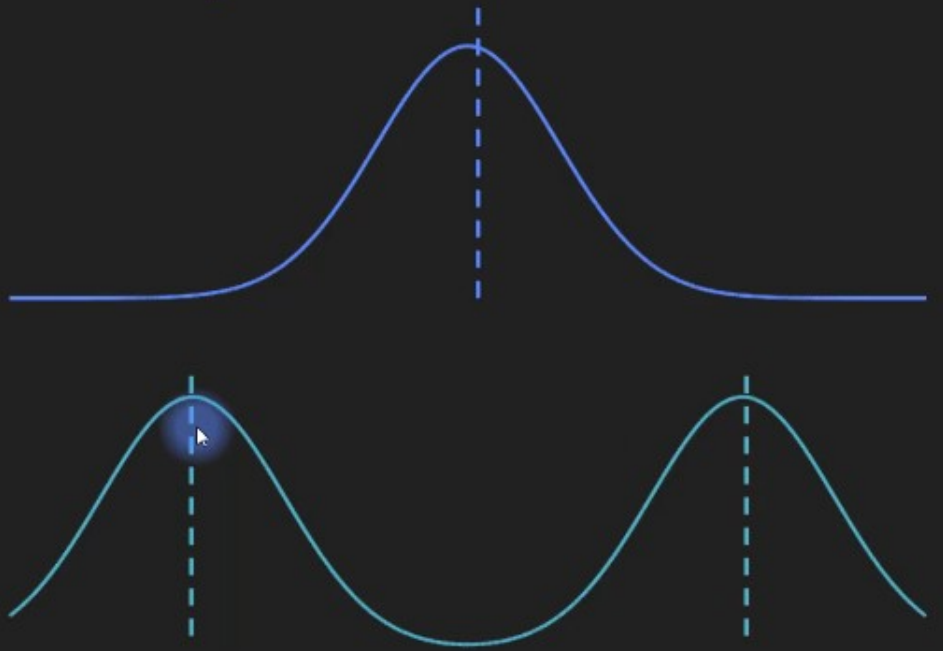
Mode

Formula:
Most common value

Suitable for:
Any distribution.

Suitable data types:
Any (numerical data should be converted to discrete)

Examples:



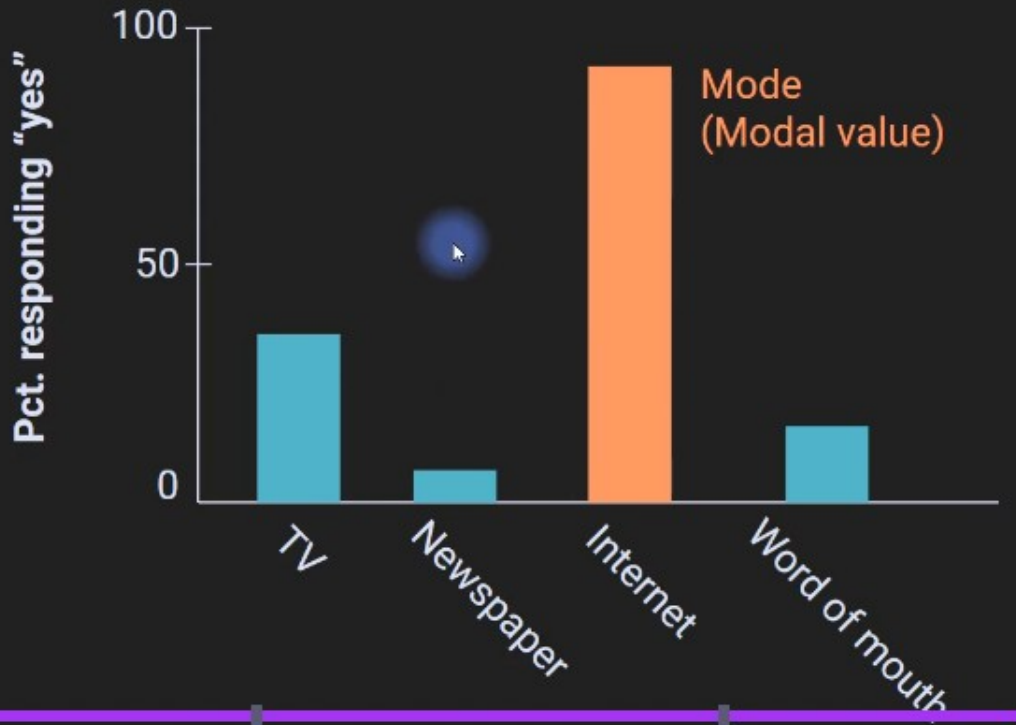
udemy

Mode

Formula:
Most common value

Suitable for:
Any distribution.

Suitable data types:
Any (numerical data should be converted to discrete)



Summary

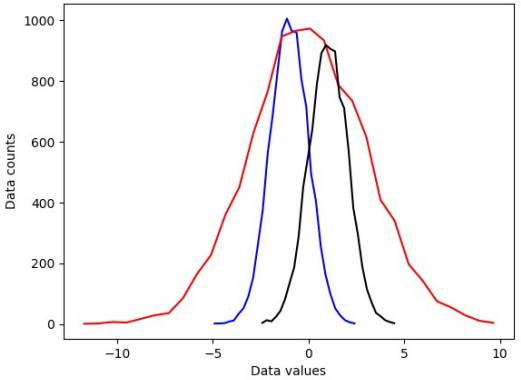
Mean: Average value, sensitive to outliers. *Most commonly used measure of central tendency.*

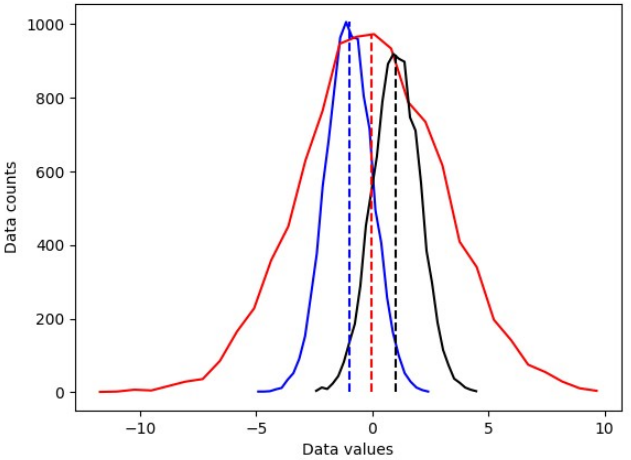
Median: Middle value (50% below, 50% above).

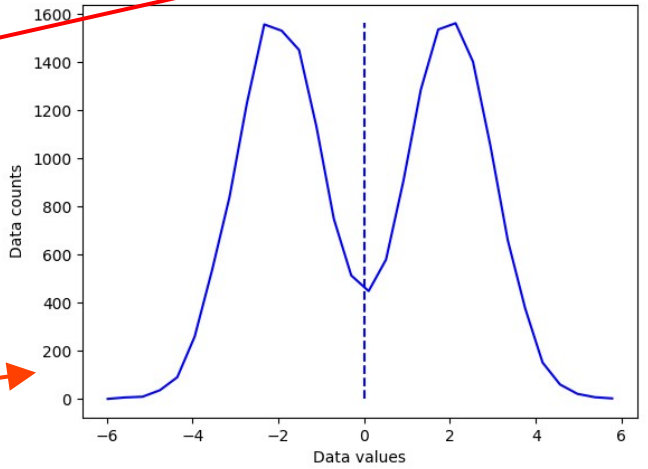
Mode: Most common value.

udemy

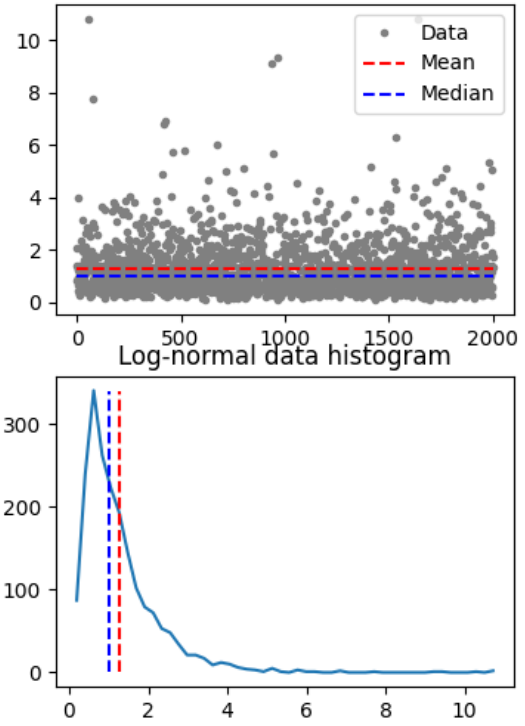
```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
## create some data distributions
# the distributions
N = 10001   # number of data points
nbins = 30  # number of histogram bins
d1 = np.random.randn(N) - 1
d2 = 3*np.random.randn(N)
d3 = np.random.randn(N) + 1
# need their histograms
y1,x1 = np.histogram(d1,nbins)
x1 = (x1[1:]+x1[:-1])/2
y2,x2 = np.histogram(d2,nbins)
x2 = (x2[1:]+x2[:-1])/2
y3,x3 = np.histogram(d3,nbins)
x3 = (x3[1:]+x3[:-1])/2
# plot them
plt.plot(x1,y1,'b')
plt.plot(x2,y2,'r')
plt.plot(x3,y3,'k')
plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()

## overlay the mean
# compute the means
mean_d1 = sum(d1) / len(d1)
mean_d2 = np.mean(d2)
mean_d3 = np.mean(d3)
# plot them
plt.plot(x1,y1,'b', x2,y2,'r', x3,y3,'k')
plt.plot([mean_d1,mean_d1],[0,max(y1)],'b--')
plt.plot([mean_d2,mean_d2],[0,max(y2)],'r--')
plt.plot([mean_d3,mean_d3],[0,max(y3)],'k--')
plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()

## "failure" of the mean
# new dataset of distribution combinations
d4 = np.hstack( (np.random.randn(N)-2,np.random.randn(N)+2) )
# and its histogram
[y4,x4] = np.histogram(d4,nbins)
x4 = (x4[:-1]+x4[1:])/2
# and its mean
mean_d4 = np.mean(d4)
plt.plot(x4,y4,'b')
plt.plot([mean_d4,mean_d4],[0,max(y4)],'b--')
plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()
```







```python
## median
# create a log-normal distribution
shift   = 0
stretch = .7
n       = 2000
nbins   = 50
# generate data
data = stretch*np.random.randn(n) + shift
data = np.exp( data )
# and its histogram
y,x = np.histogram(data,nbins)
x = (x[:-1]+x[1:])/2
# compute mean and median
datamean = np.mean(data)
datamedian = np.median(data)
# plot data
fig,ax = plt.subplots(2,1,figsize=(4,6))
ax[0].plot(data,'.',color=[.5,.5,.5],label='Data')
ax[0].plot([1,n],[datamean,datamean],'r--',label='Mean')
ax[0].plot([1,n],[datamedian,datamedian],'b--',label='Median')
ax[0].legend()
ax[1].plot(x,y)
ax[1].plot([datamean,datamean],[0,max(y)],'r--')
ax[1].plot([datamedian,datamedian],[0,max(y)],'b--')
ax[1].set_title('Log-normal data histogram')
plt.show()

## mode
data = np.round(np.random.randn(10))
uniq_data = np.unique(data)
for i in range(len(uniq_data)):
 print(f'{uniq_data[i]} appears {sum(data==uniq_data[i])} times.')
print(' ')
#print('The modal value is %g'%stats.mode(data)[0][0])
print('The modal value is %g'%stats.mode(data)[0])
```



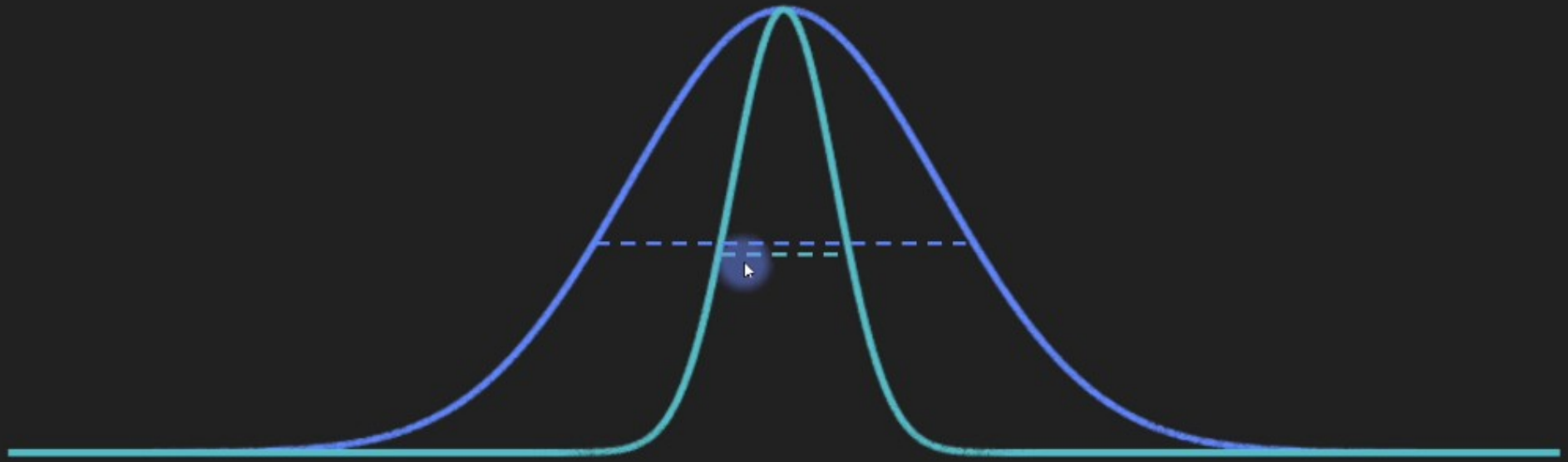

```
-1.0 appears 3 times.
-0.0 appears 6 times.
1.0 appears 1 times.

The modal value is -0
```

Measures of dispersion

In this video, you will learn

- What the term “dispersion” means.
- The formulas for the two main measures of dispersion (standard deviation, variance).
- Introduction to related measures (Fano, CV).

Concept of dispersion



Variance

Formula: $\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$

Suitable for:
Any distribution.

Suitable data types:
Numerical
Ordinal (but requires mean...)

Example:

x = [8,0,4,1,-2,7]

var(x) = mean([5,-3,1,-2,-5,4]^2)

Variance

Formula: $\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$

Not formally the average because we divide by N-1, not N.

Apologies for the confusion.

Example:

Suitable for:
Any distribut

$x = [\ 8,0,4,1,-2,7\ ]$

Suitable data
Numerical
Ordinal (but requires mean...)

$var(x) = mean([\ 5,-3,1,-2,-5,4\ ]^2)$
$var(x) = 16$

Variance

Formula: $\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$

[8,0,4,1,-2,7], σ^2= 16

Suitable for:
Any distribution.

[2,3,4,3,4,4], σ^2=0.67

Suitable data types:
Numerical
Ordinal (but requires mean...)

Some questions about the formula

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

Why mean-center?

Variance indicates the dispersion around the *average*;
the following two datasets should have the same variance:

d1 = [1 2 3 3 2 1]
d2 = [101 102 103 103 102 101]

Some questions about the formula

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

Why are the differences squared?

We want the *distances* to the average;
without squaring the variance would be 0.

d1 = [1 2 3 3 2 1]
Mean-centered d1 = [-1 0 1 1 0 -1] ⇨ sums to 0!

Some questions about the formula

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}|x_i - \overline{x}|$$

Why not take the absolute value (“mean absolute difference”)?

Squaring: emphasizes large values; is better for optimization (continuous and differentiable); is closer to Euclidean distance; is the second “moment” of the distribution; better link to least-squares regression; other nice properties…

MAD: Also good; robust to outliers; less commonly used.

Some questions about the formula

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

Why divide by n-1?

Dividing by N-1 is for *sample variance*;
Dividing by N is for *population variance*.

Population mean is a *theoretical quantity*, whereas the sample mean is an *empirical quantity*.

Some questions about the formula

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2$$

Why divide by n-1?

The population mean of a die is 3.5.
Roll a die 4 times (sample). *The sample mean is 3.*

How die rolls do you need to see to know all 4 values?
Answer: 3. For example: 1, 2, 4, ?

18 people have written a note here.

Hence, there are N-1 free values, or degrees of freedom.

Standard deviation

Formula: $\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2}$

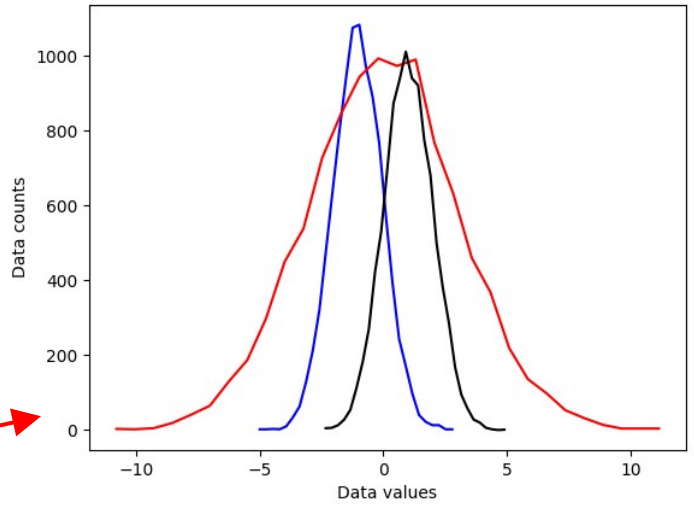
$$= \left(\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2\right)^{1/2}$$

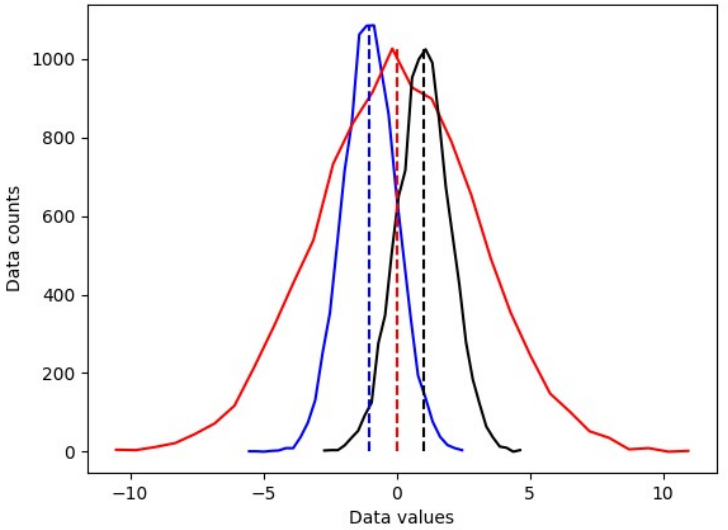
Two other related measures

Fano factor: $F = \frac{\sigma^2}{\mu}$

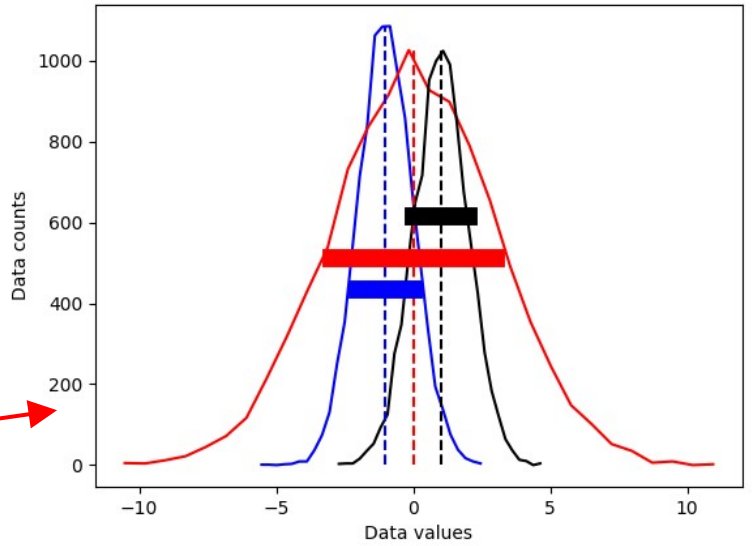
Coefficient of variation: $CV = \frac{\sigma}{\mu}$

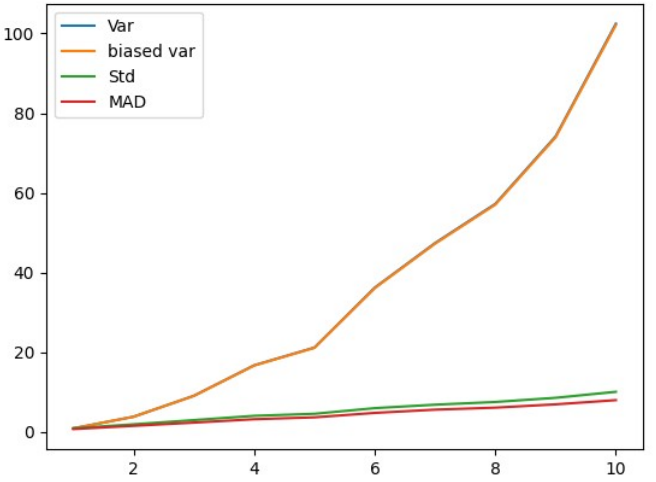
```python
import matplotlib.pyplot as plt
import numpy as np
## create some data distributions
# the distributions
N = 10001   # number of data points
nbins = 30  # number of histogram bins
d1 = np.random.randn(N) - 1
d2 = 3*np.random.randn(N)
d3 = np.random.randn(N) + 1
# need their histograms
y1,x1 = np.histogram(d1,nbins)
x1 = (x1[1:]+x1[:-1])/2
y2,x2 = np.histogram(d2,nbins)
x2 = (x2[1:]+x2[:-1])/2
y3,x3 = np.histogram(d3,nbins)
x3 = (x3[1:]+x3[:-1])/2
# plot them
plt.plot(x1,y1,'b')
plt.plot(x2,y2,'r')
plt.plot(x3,y3,'k')
plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()

# side note:
meanval = 10.2
stdval  = 7.5
numsamp = 123
# this
np.random.normal(meanval,stdval,numsamp)
# is equivalent to
np.random.randn(numsamp)*stdval + meanval

## overlay the mean

# compute the means
mean_d1 = sum(d1) / len(d1)
mean_d2 = np.mean(d2)
mean_d3 = np.mean(d3)
# plot them
plt.plot(x1,y1,'b', x2,y2,'r', x3,y3,'k')
plt.plot([mean_d1,mean_d1],[0,max(y1)],'b--')
plt.plot([mean_d2,mean_d2],[0,max(y2)],'r--')
plt.plot([mean_d3,mean_d3],[0,max(y3)],'k--')

plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()
```



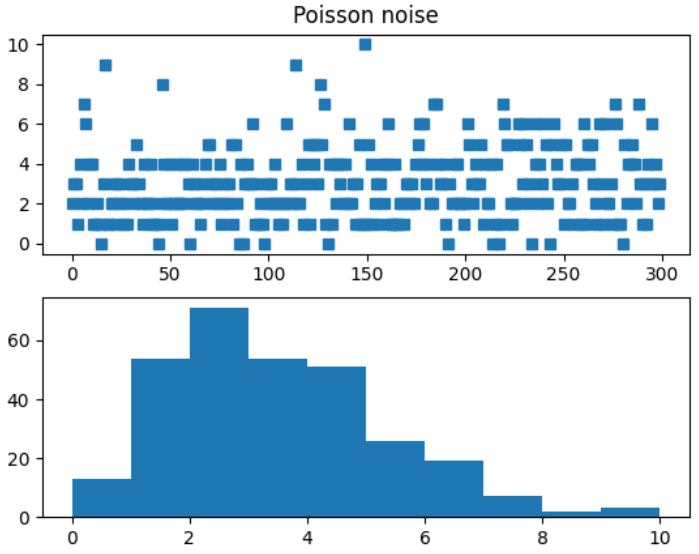


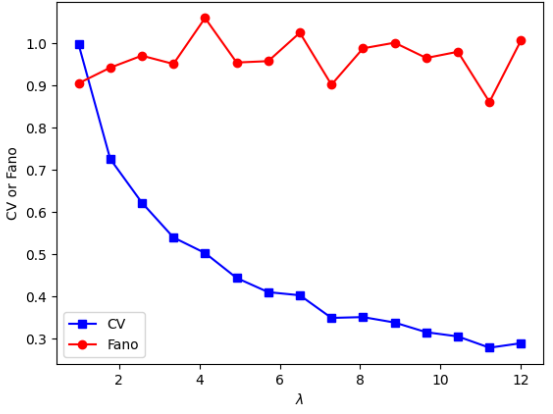
```
## now for the standard deviation
# initialize
stds = np.zeros(3)
# compute standard deviations
stds[0] = np.std(d1,ddof=1)
stds[1] = np.std(d2,ddof=1)
stds[2] = np.std(d3,ddof=1)
# same plot as earlier
plt.plot(x1,y1,'b', x2,y2,'r', x3,y3,'k')
plt.plot([mean_d1,mean_d1],
         [0,max(y1)], 'b--',
         [mean_d2,mean_d2], [0,max(y2)],'r--',
         [mean_d3,mean_d3],[0,max(y3)],'k--')
# now add stds
plt.plot([mean_d1-stds[0],mean_d1+stds[0]],
         [.4*max(y1),.4*max(y1)],'b',linewidth=10)
plt.plot([mean_d2-stds[1],mean_d2+stds[1]],
         [.5*max(y2),.5*max(y2)],'r',linewidth=10)
plt.plot([mean_d3-stds[2],mean_d3+stds[2]],
         [.6*max(y3),.6*max(y3)],'k',linewidth=10)
plt.xlabel('Data values')
plt.ylabel('Data counts')
plt.show()

## different variance measures
variances = np.arange(1,11)
N = 300
varmeasures = np.zeros((4,len(variances)))
for i in range(len(variances)):
    # create data and mean-center
    data = np.random.randn(N) * variances[i]
    datacent = data - np.mean(data)
    # variance
    varmeasures[0,i] = sum(datacent**2) / (N-1)
    # "biased" variance
    varmeasures[1,i] = sum(datacent**2) / N
    # standard deviation
    varmeasures[2,i] = np.sqrt( sum(datacent**2) / (N-1) )
    # MAD (mean absolute difference)
    varmeasures[3,i] = sum(abs(datacent)) / (N-1)
# show them!
plt.plot(variances,varmeasures.T)
plt.legend(('Var','biased var','Std','MAD'))
plt.show()
```





```python
## Fano factor and coefficient of variation (CV)
# need positive-valued data (why?)
data = np.random.poisson(3,300)  # "Poisson noise"
fig,ax = plt.subplots(2,1)
ax[0].plot(data,'s')
ax[0].set_title('Poisson noise')
ax[1].hist(data)
plt.show()


## compute fano factor and CV for a range of lambda parameters
# list of parameters
lambdas = np.linspace(1,12,15)
# initialize output vectors
fano = np.zeros(len(lambdas))
cv   = np.zeros(len(lambdas))

for li in range(len(lambdas)):
    # generate new data
    data = np.random.poisson(lambdas[li],1000)
    # compute the metrics
    cv[li]   = np.std(data) / np.mean(data) # need ddof=1 here?
    fano[li] = np.var(data) / np.mean(data)

## and plot
plt.plot(lambdas,cv,'bs-')
plt.plot(lambdas,fano,'ro-')
plt.legend(('CV','Fano'))
plt.xlabel('$\lambda$')
plt.ylabel('CV or Fano')
plt.show()

~
```



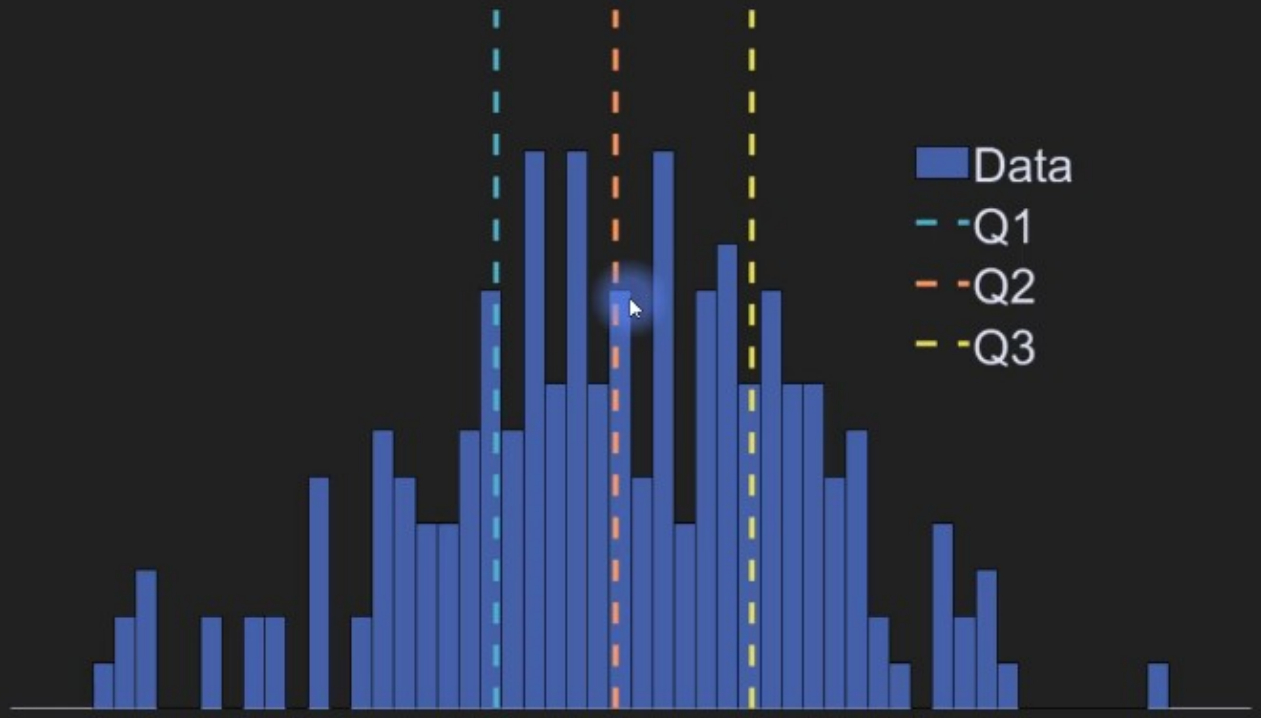


Interquartile range (IQR)

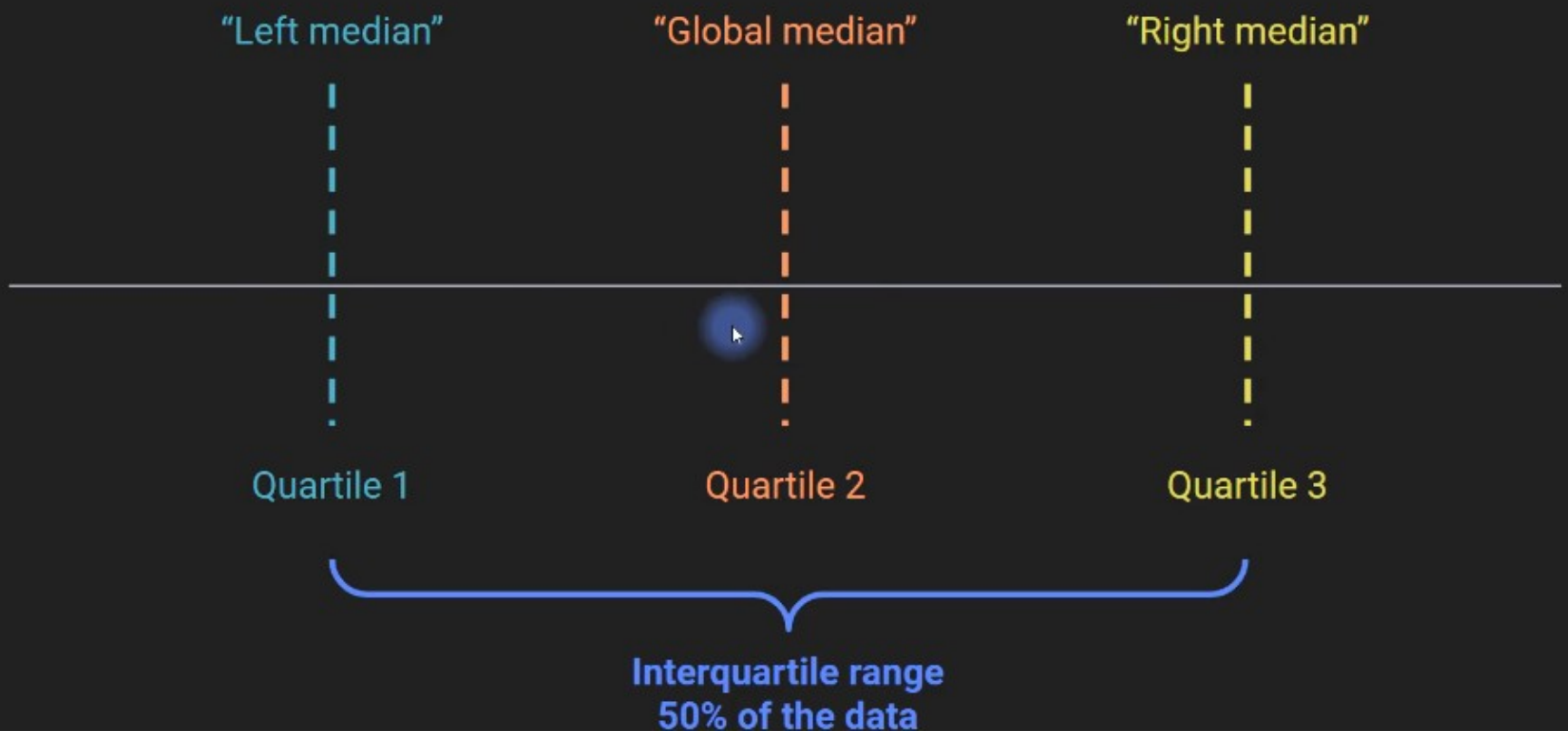
In this video, you will learn

- How to compute IQR.
- Why IQR is a useful complement to the median.

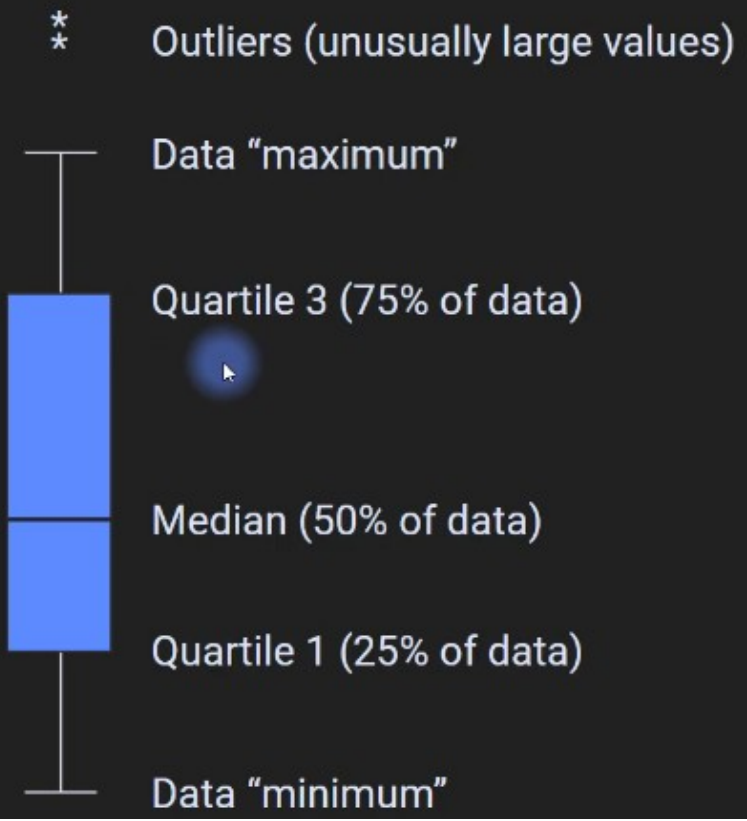
Medians, super-charged



Medians, super-charged



The box-and-whisker plot, redux



```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## create the data
# random number data
n = 1000
data = np.random.randn(n)**2
# rank-transform the data and scale to 1
dataR = stats.rankdata(data)/n
# find the values closest to 25% and 75% of the distribution
q1 = np.argmin((dataR-.25)**2)
q3 = np.argmin((dataR-.75)**2)
# get the two values in the data
iq_vals = data[[q1,q3]]
# IQR is the difference between them
iqrange1 = iq_vals[1] - iq_vals[0]
# or use Python's built-in function ;)
iqrange2 = stats.iqr(data)
print(iqrange1,iqrange2)
```

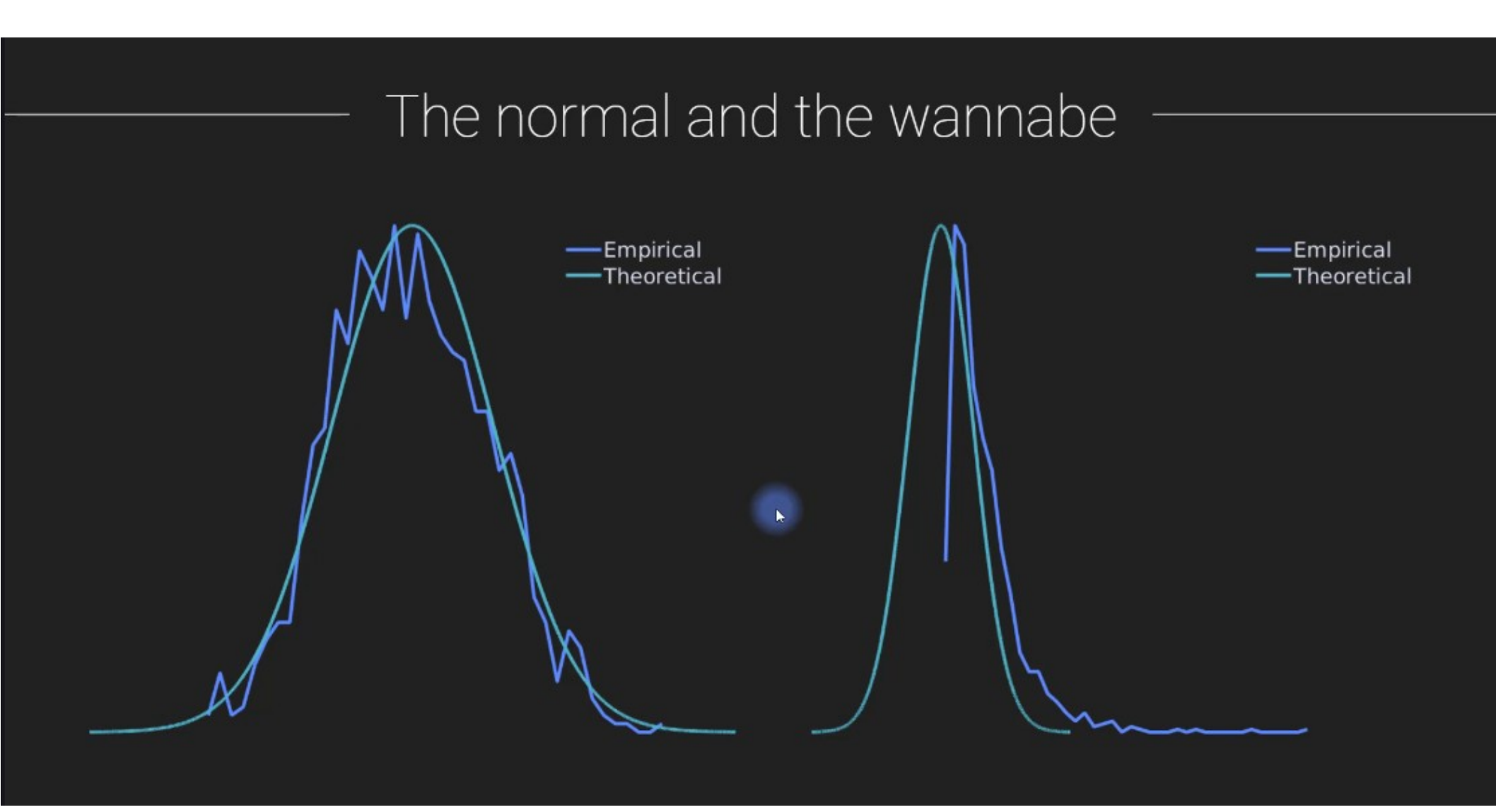
```
1.1442409717366837 1.143973414651711
```

QQ plots (QQ = quintile-quintile)

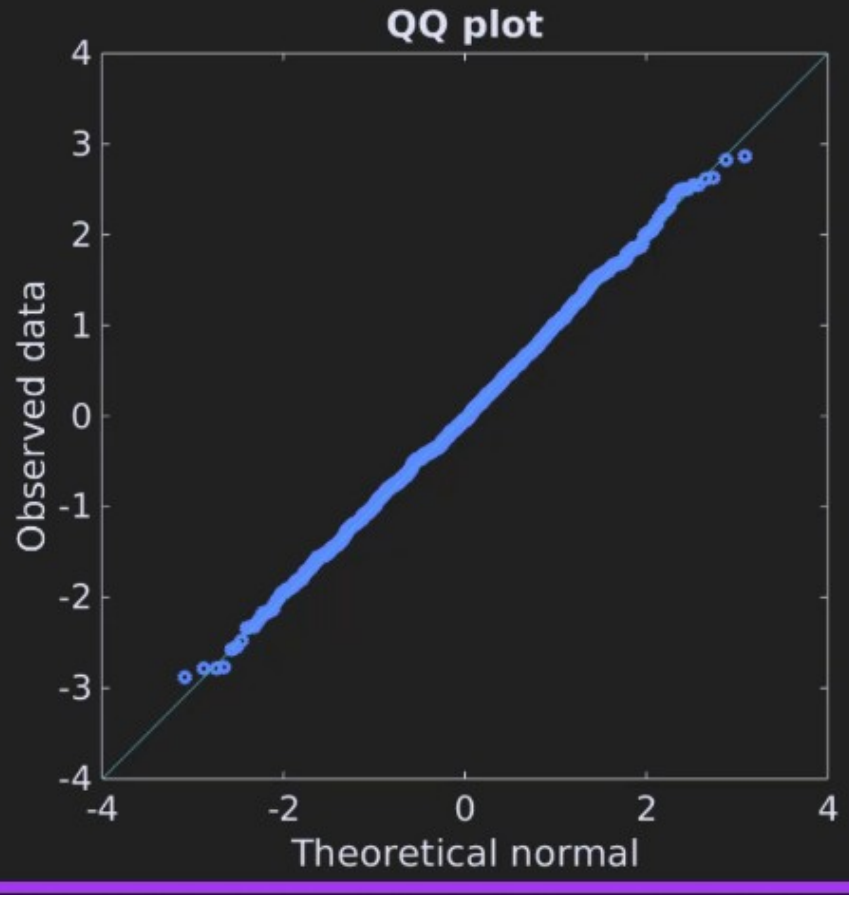
In this video, you will learn:

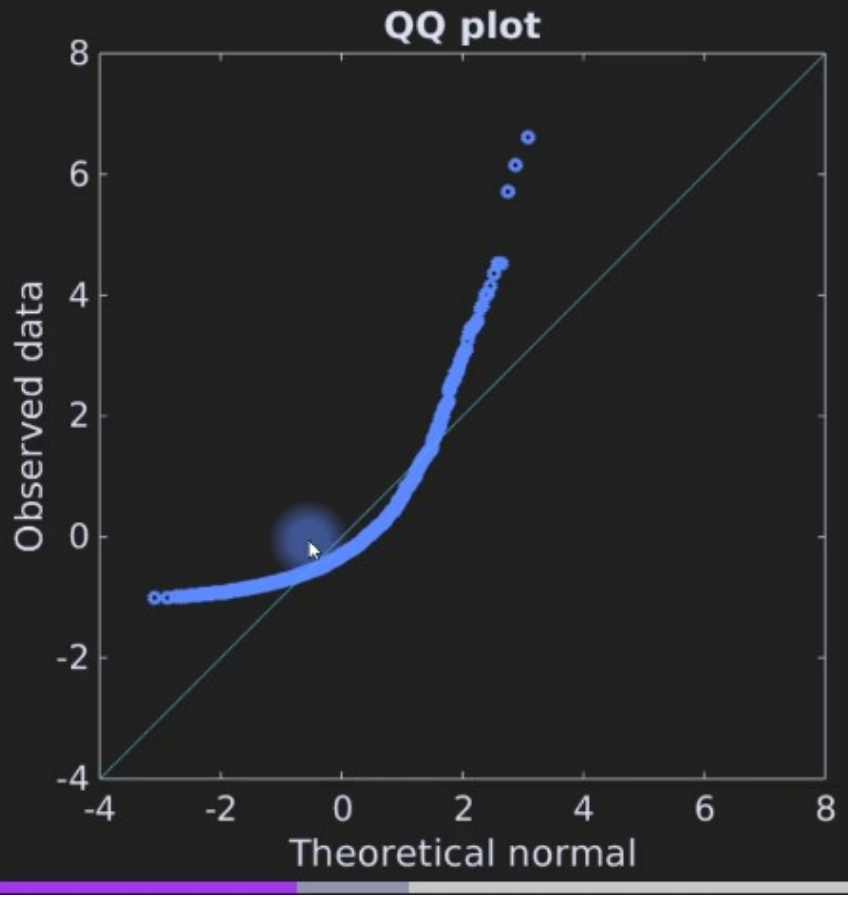
- how to interpret QQ plots.

The normal and the wannabe

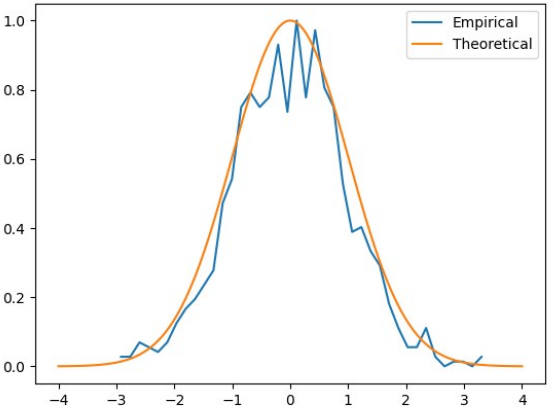


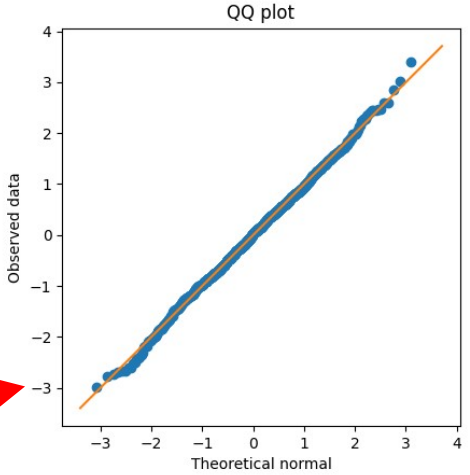
The QQ plot

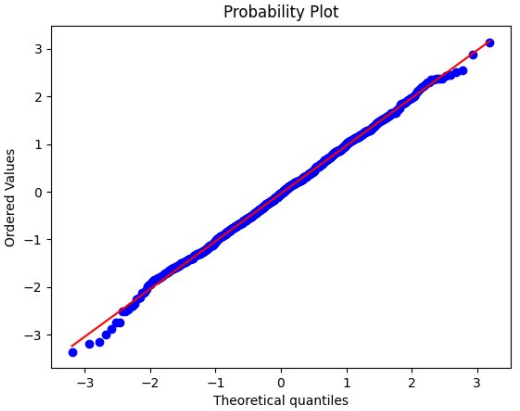

QQ plot
Observed data
Theoretical normal
4
3
2
1
0
-1
-2
-3
-4
-4
-2
0
2
4


QQ plot
Observed data
Theoretical normal
8
6
4
2
0
-2
-4
-4
-2
0
2
4
6
8

```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate data
n    = 1000
data = np.random.randn(n)
# data = np.exp( np.random.randn(n)*.8 ) # log-norm distribution
# theoretical normal distribution given N
x = np.linspace(-4,4,10001)
theonorm = stats.norm.pdf(x)
theonorm = theonorm/max(theonorm)
# plot histograms on top of each other
yy,xx = np.histogram(data,40)
yy = yy/max(yy)
xx = (xx[:-1]+xx[1:])/2
plt.plot(xx,yy,label='Empirical')
plt.plot(x,theonorm,label='Theoretical')
plt.legend()
plt.show()

## create a QQ plot
zSortData  = np.sort(stats.zscore(data))
sortNormal = stats.norm.ppf(np.linspace(0,1,n))
# QQ plot is theory vs reality
plt.plot(sortNormal,zSortData,'o')
# set axes to be equal
xL,xR = plt.xlim()
yL,yR = plt.ylim()
lims  = [ np.min([xL,xR,yL,yR]),np.max([xL,xR,yL,yR]) ]
plt.xlim(lims)
plt.ylim(lims)
# draw red comparison line
plt.plot(lims,lims)
plt.xlabel('Theoretical normal')
plt.ylabel('Observed data')
plt.title('QQ plot')
plt.axis('square')
plt.show()


## Python solution
x = stats.probplot(data,plot=plt)
plt.show()
```







Statistical moments

In this video, you will learn

- What a “moment” means in statistics parlance.
- How the various moments are related to each other.
- The first four statistical moments.

Unstandardized statistical moments

General formula $$m_k = n^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^k$$

First moment: mean $$m_1 = n^{-1} \sum_{i=1}^{n} x_i$$

Second moment: variance $$m_2 = n^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^2$$

Standardized statistical moments

Mean: Average value

$$m_1 = n^{-1} \sum_{i=1}^{n} x_i$$

Variance: Dispersion

$$m_2 = n^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^2$$

Skewness: Dispersion asymmetry

$$m_3 = (n\sigma^3)^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^3$$

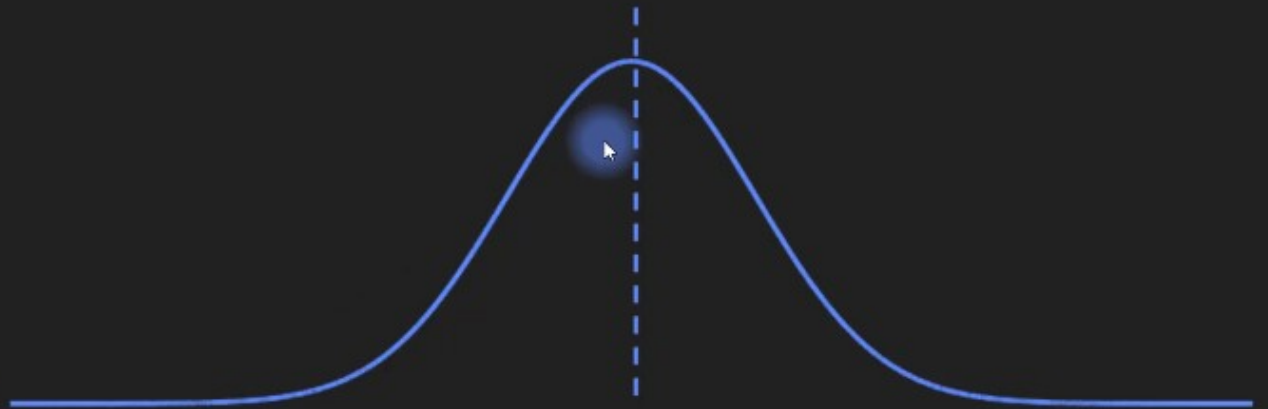
Kurtosis: Tail "fatness"

$$m_4 = (n\sigma^4)^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^4$$

First moment: mean

$$m_1 = n^{-1} \sum_{i=1}^{n} x_i$$

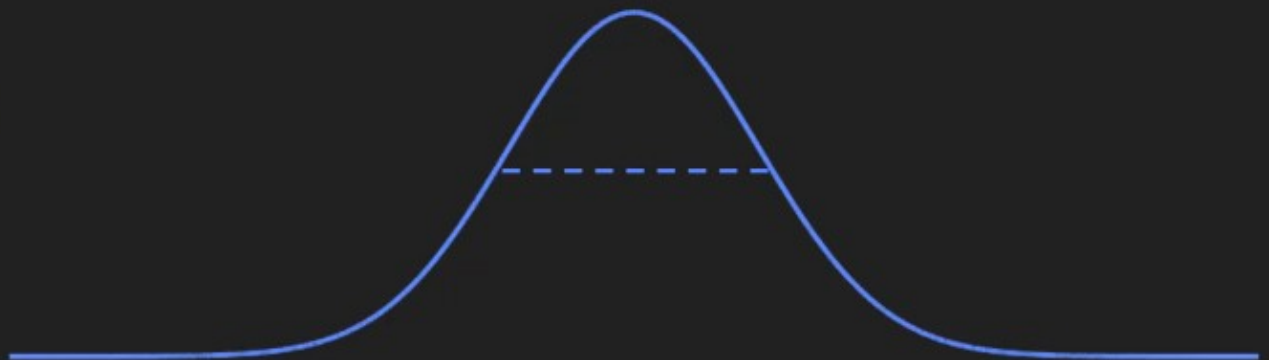
Mean: Average value



Second moment: variance

$$m_2 = n^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^2$$

Variance: Dispersion



Third moment: skewness

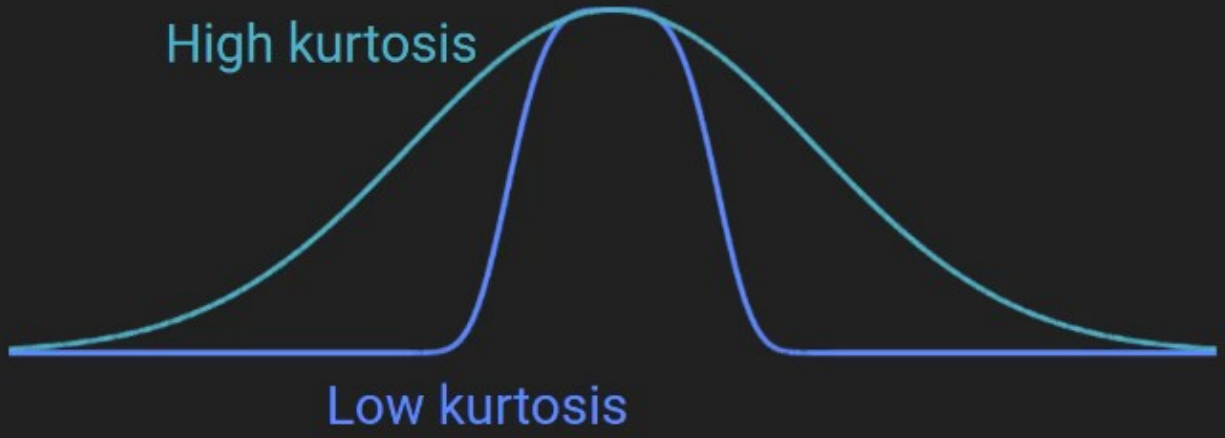
$$m_3 = (n\sigma^3)^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^3$$

Skewness: Dispersion asymmetry

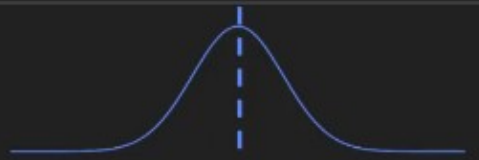
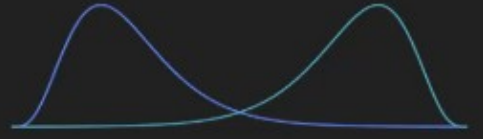
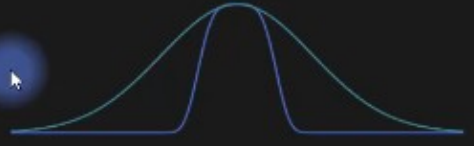
Fourth moment: kurtosis

$$m_4 = (n\sigma^4)^{-1} \sum_{i=1}^{n} (x_i - \overline{x})^4$$

Kurtosis: Tail "fatness"



What you should memorize

Moment number	Stats name	Interpretation	Picture
First moment	Mean	Average value	
Second moment	Variance	Dispersion	
Third moment	Skewness	Dispersion asymmetry	
Fourth moment	Kurtosis	Tail “fatness”	

Histograms part 2: Number of bins

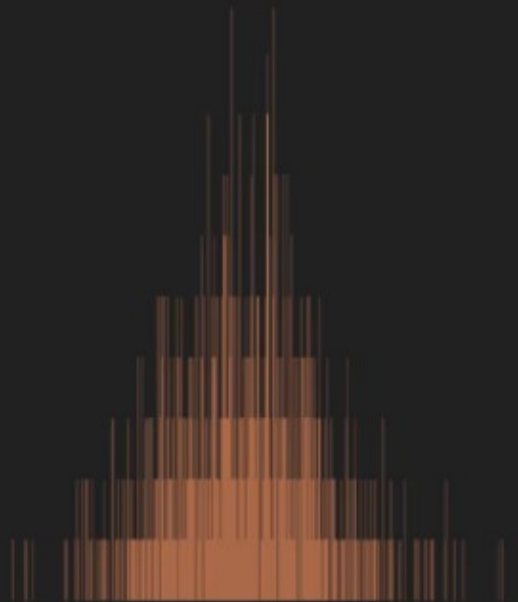
In this video, you will learn

- What “bins” mean in the context of histograms.
- Why bin size is important when creating histograms.
- Several “rules” (actually guidelines) for determining the number of histogram bins.

Histogram bins

4 bins

30 bins



700 bins

How many bins?

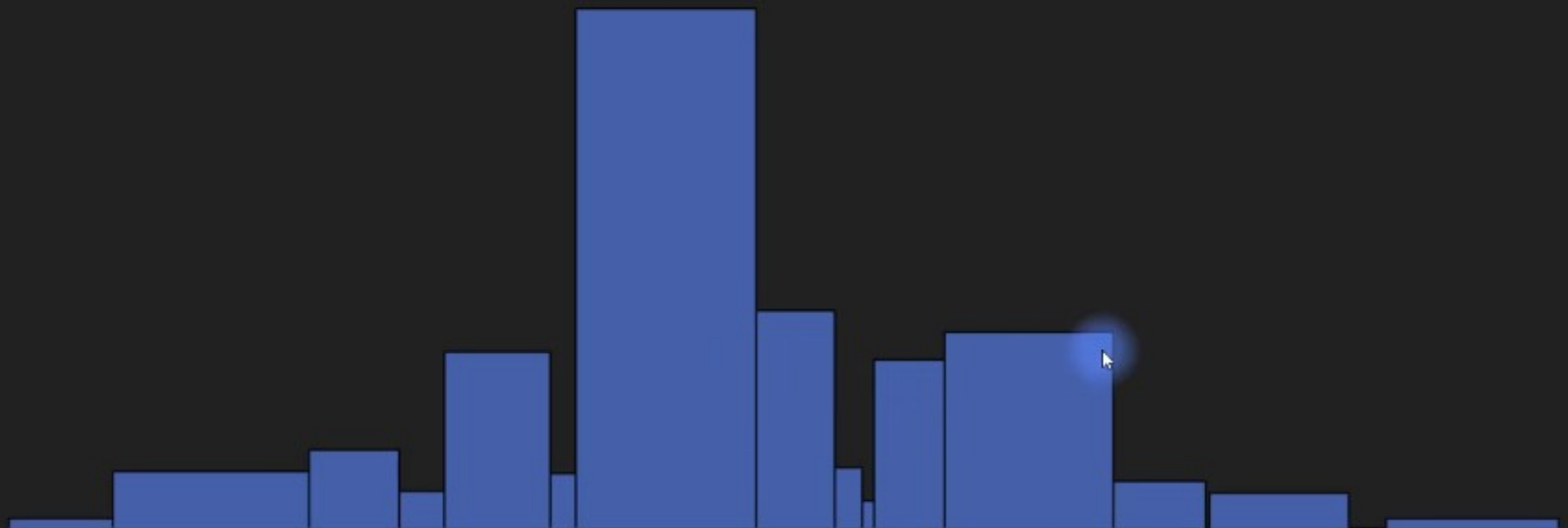
$$k = \left\lceil \frac{\max(x) - \min(x)}{h} \right\rceil$$

How many bins?

Guideline	Formula	Key advantage
Sturges	$k = \lceil \log 2(n) \rceil + 1$	Depends on data count.
Freedman-Diaconis	$h = 2 \frac{\mathrm{IQR}}{\sqrt[3]{n}}$	Depends on data count and on data spread.
Arbitrary	$k = 40$	Easy to use.

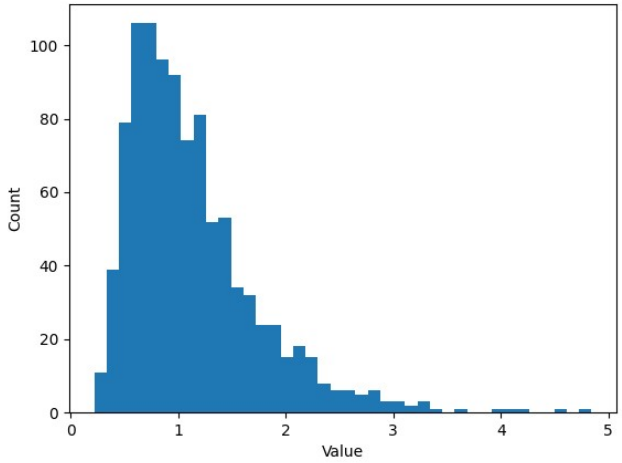
Usually the best one. Probably the most common choice.

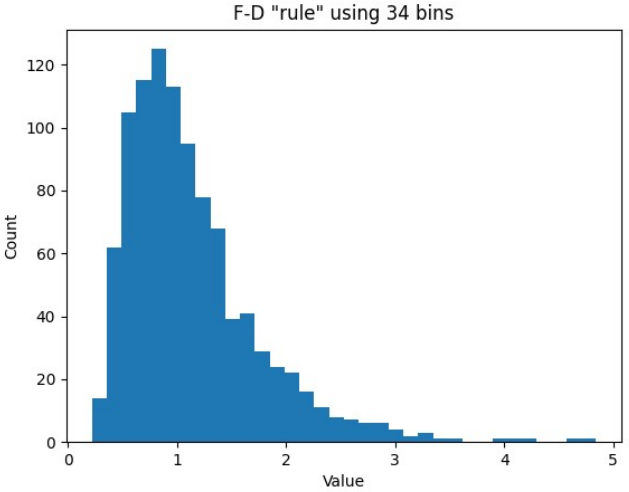
What about variable bin sizes?

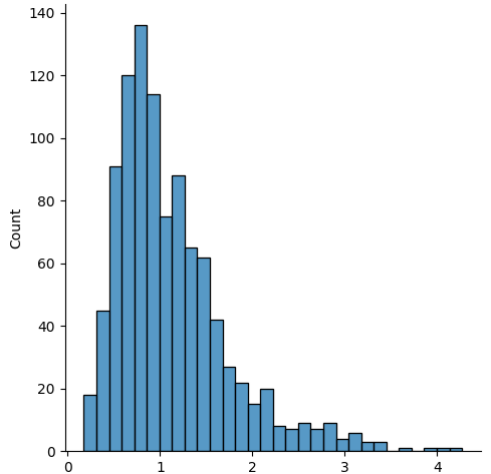


Conclusion: Looks pretty cool, but don't do it.

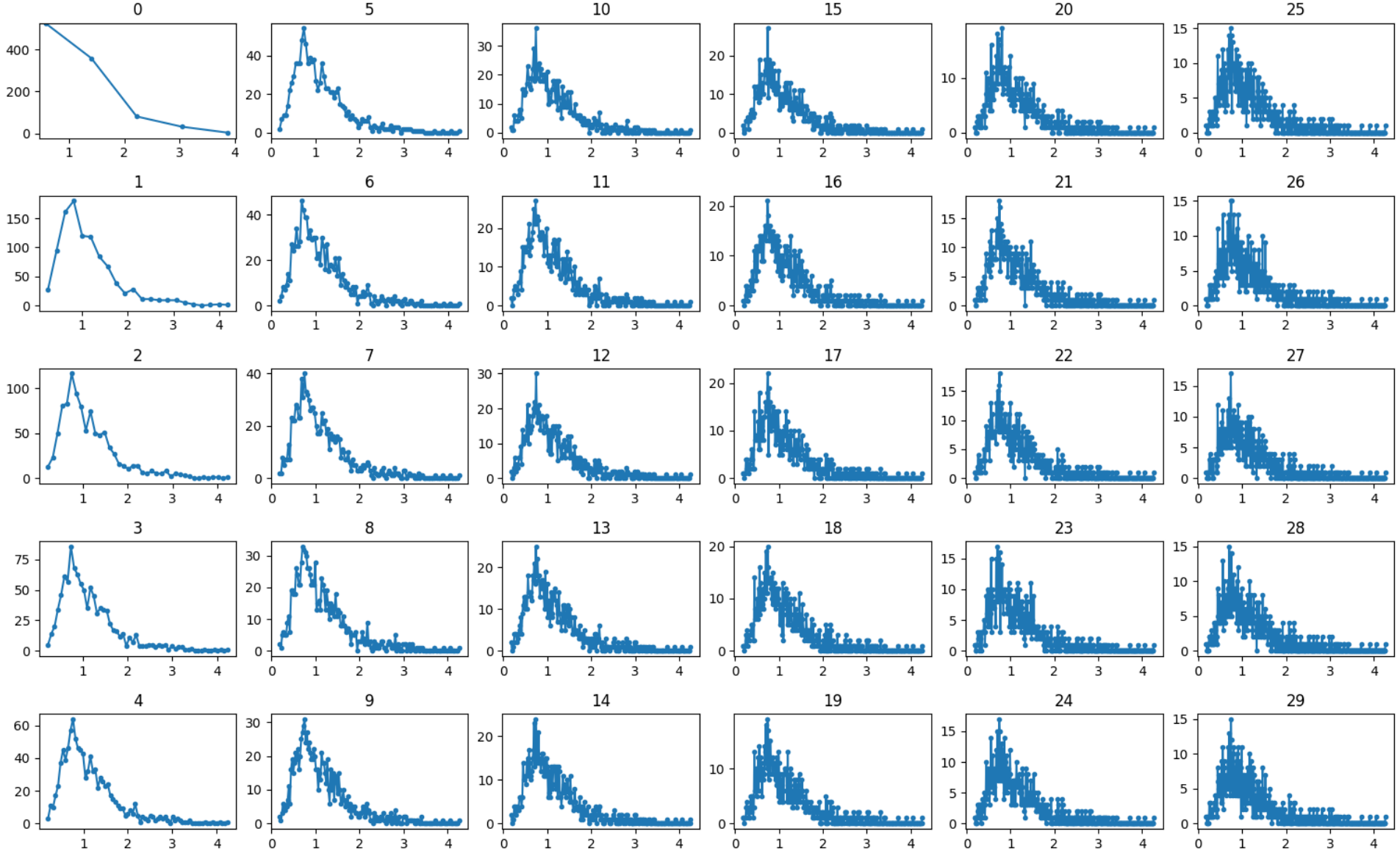
```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## create some data
# number of data points
n = 1000
# number of histogram bins
k = 40
# generate log-normal distribution
data = np.exp( np.random.randn(n)/2 )
# one way to show a histogram
plt.hist(data,k)
plt.xlabel('Value')
plt.ylabel('Count')
plt.show()

## try the Freedman-Diaconis rule
r = 2*stats.iqr(data)*n**(-1/3)
b = np.ceil( (max(data)-min(data) )/r )
plt.hist(data,int(b))
# or directly from the hist function
#plt.hist(data,bins='fd')
plt.xlabel('Value')
plt.ylabel('Count')
plt.title('F-D "rule" using %g bins'%b)
plt.show()

# small aside on Seaborn
import seaborn as sns
sns.displot(data) # uses FD rule by default

## lots of histograms with increasing bins
bins2try = np.round( np.linspace(5,n/2,30) )
fig, axs = plt.subplots(5,int(len(bins2try)/5))
for bini in range(len(bins2try)):
    i,j=int(bini%5),int(bini/5)
    y,x = np.histogram(data,int(bins2try[bini]))
    x = (x[:-1]+x[1:])/2
    axs[i,j].plot(x,y,'.-')
    axs[i,j].set_title(str(bini))

plt.show()
```







SONRAKİ SAYFA


0
5
10
15
20
25
1
6
11
16
21
26
2
7
12
17
22
27
3
8
13
18
23
28
4
9
14
19
24
29

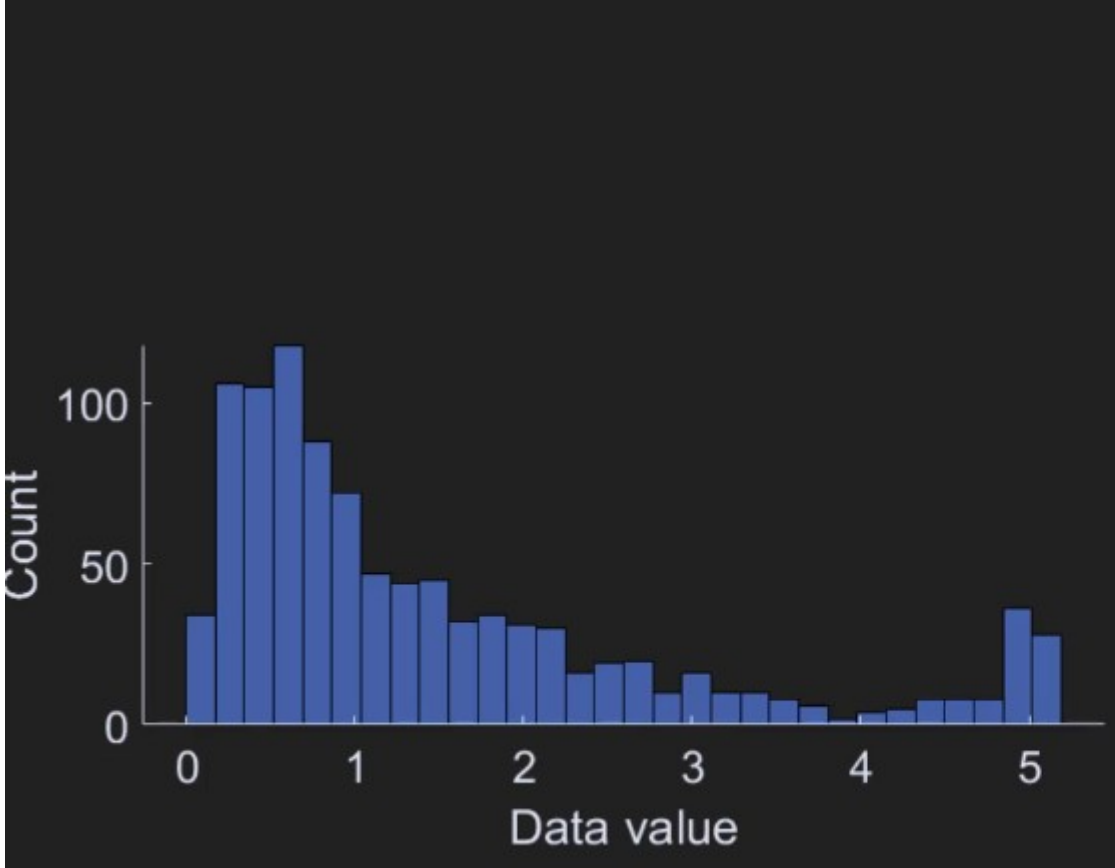
Violin plots

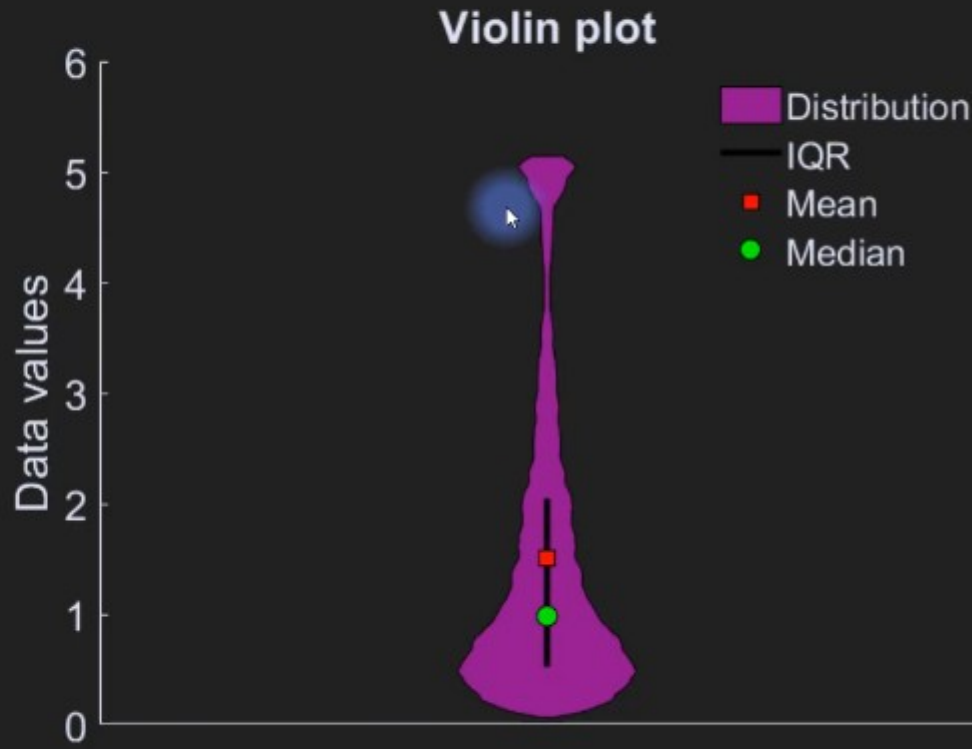
In this video, you will learn

- What a violin plot is and how to interpret one.
- How to create a violin plot.

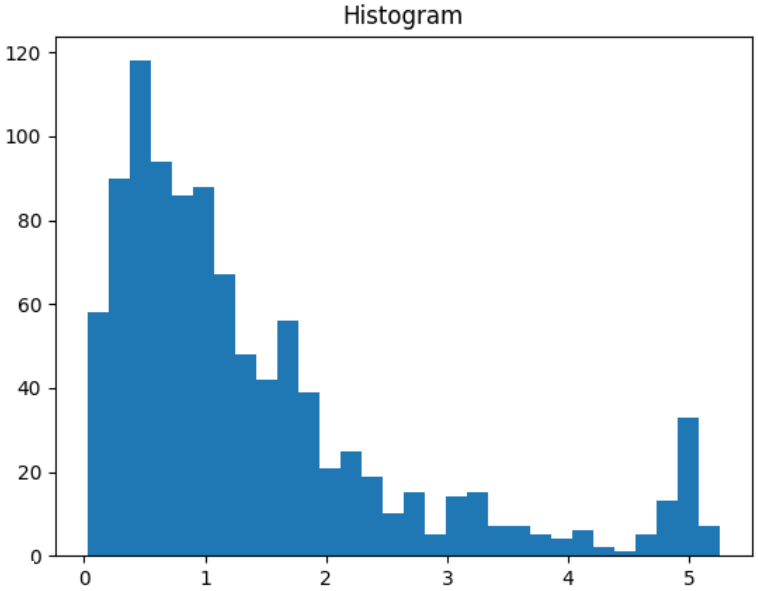
A violin made from histograms

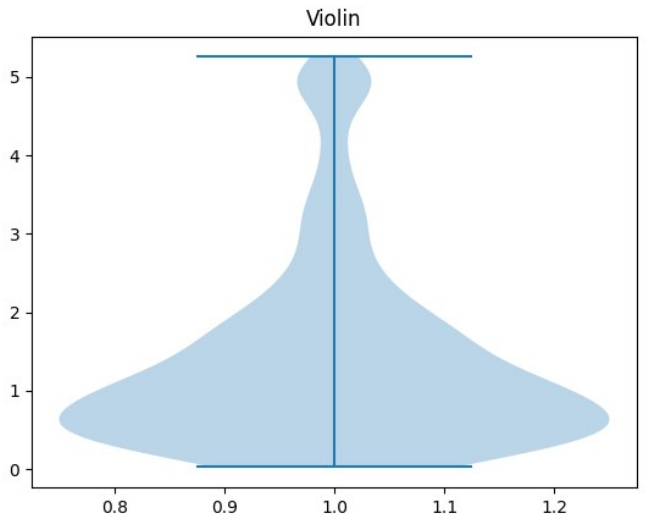
A violin made from histograms





```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## create the data
n = 1000
thresh = 5 # threshold for cropping data
data = np.exp( np.random.randn(n) )
data[data>thresh] = thresh + np.random.randn(sum(data>thresh))*.1
# show histogram
plt.hist(data,30)
plt.title('Histogram')
plt.show()

# show violin plot
plt.violinplot(data)
plt.title('Violin')
plt.show()

# another option: swarm plot
import seaborn as sns
sns.swarmplot(data,orient='v')
```





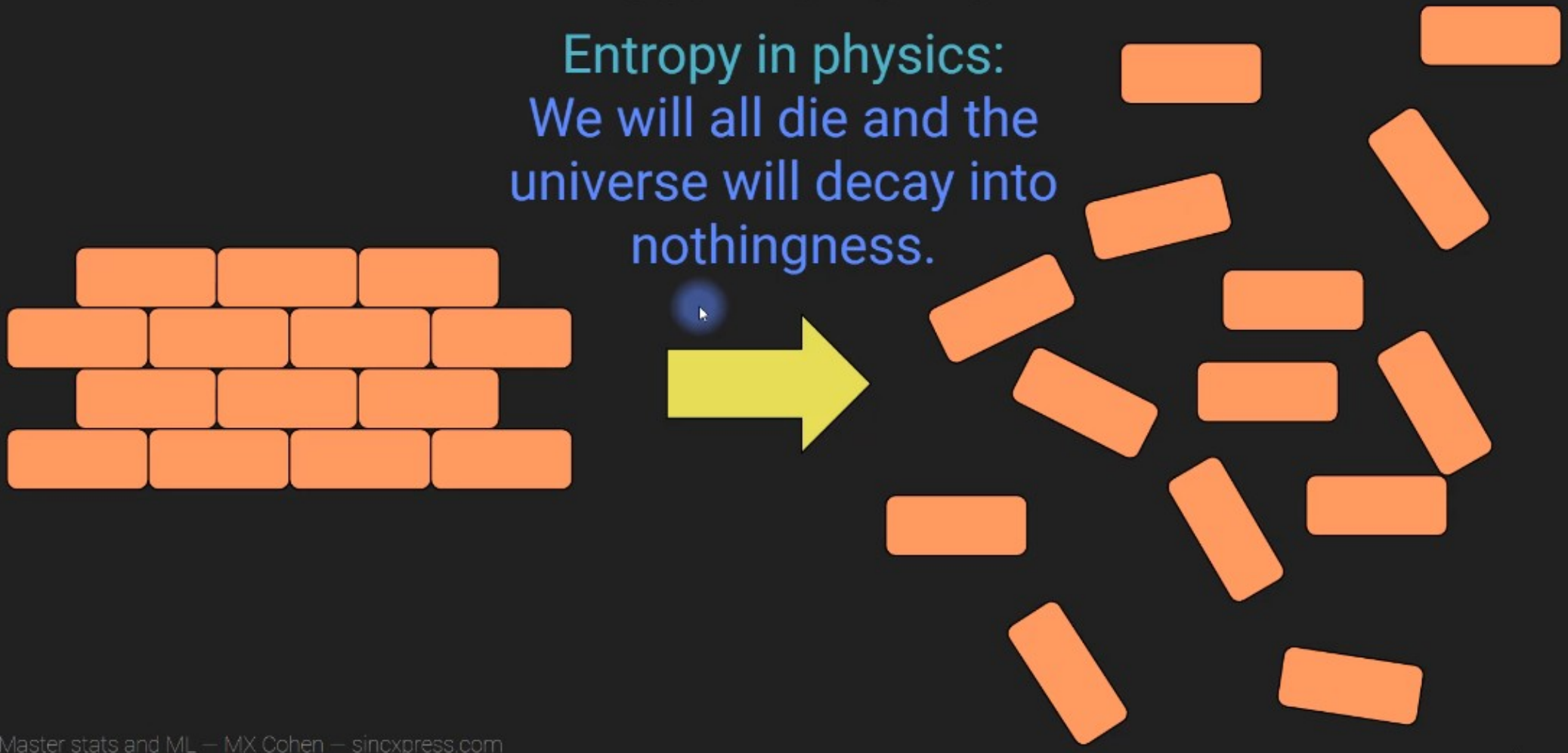
Shannon entropy

In this video, you will learn

- How to compute entropy.
- How to interpret entropy.

What's in a name?

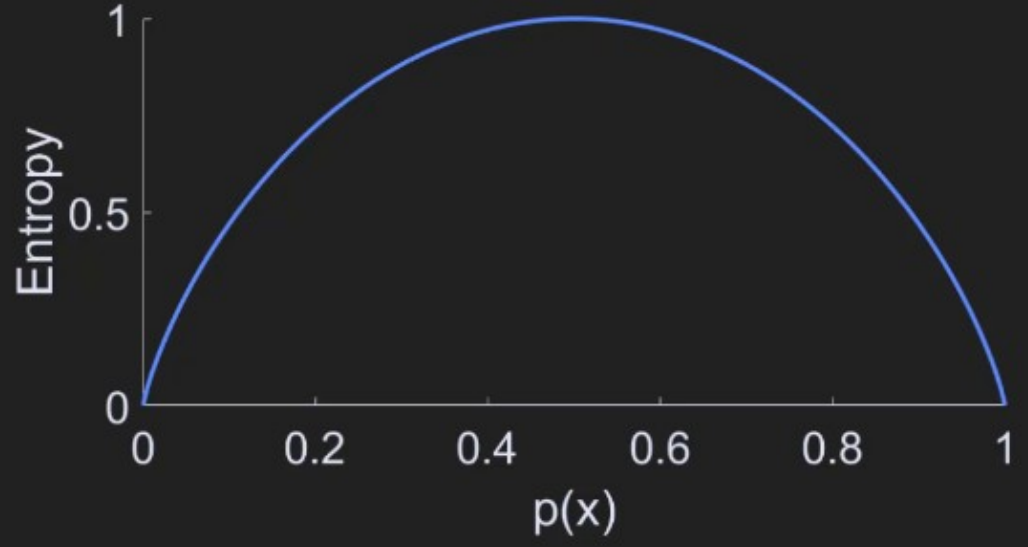
Entropy in physics:
We will all die and the universe will decay into nothingness.



What's in a name?

Entropy in information theory: Everything is great and happy, and surprising things convey more information.

https://en.wikipedia.org/wiki/Claude_Shannon



Formula for entropy

$$H = -\sum_{i=1}^{n} p(x_i) \log_2(p(x_i))$$

x = data values
p = probability

What types of data can this formula be applied to?

Nominal, ordinal, discrete

What if you have interval or ratio data?

Convert to discrete by binning (via histogram creation)

Important: Entropy depends on bin size and number!

Interpreting entropy

$$H = -\sum_{i=1}^{n} p(x_i)\log_2(p(x_i))$$

x = data values
p = probability

High entropy means that the dataset has a lot of variability.
Low entropy means that most of the values of the dataset repeat (and therefore are redundant).

How does entropy differ from variance?

Entropy is nonlinear and makes no assumptions about the distribution.

Variance depends on the validity of the mean and therefore is appropriate for roughly normal data.

Interpreting entropy

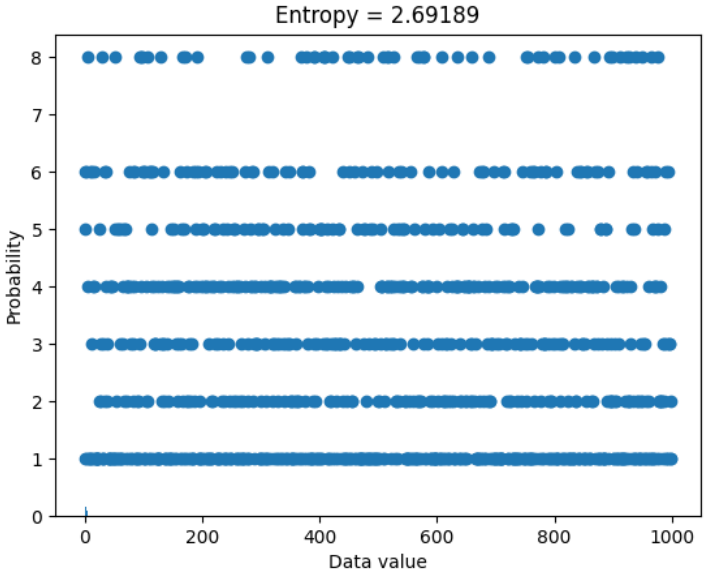
$$H = -\sum_{i=1}^{n} p(x_i) \log_2(p(x_i))$$

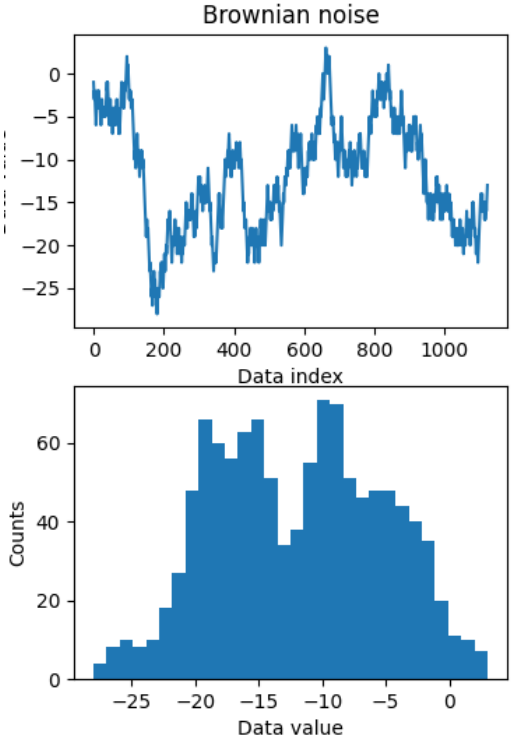
Units: “bits”

$$H = -\sum_{i=1}^{n} p(x_i) \ln(p(x_i))$$

Units: “nats”

```python
import matplotlib.pyplot as plt
import numpy as np

## "discrete" entropy
# generate data
N = 1000
numbers = np.ceil( 8*np.random.rand(N)**2 )
numbers[numbers==7] = 4
plt.plot(numbers,'o')

## "discrete" entropy
# generate data
N = 1000
numbers = np.ceil( 8*np.random.rand(N)**2 )
# get counts and probabilities
u = np.unique(numbers)
probs = np.zeros(len(u))
for ui in range(len(u)):
    probs[ui] = sum(numbers==u[ui]) / N
# compute entropy
entropee = -sum( probs*
                 np.log2(probs+np.finfo(float).eps) )

# plot
plt.bar(u,probs)
plt.title('Entropy = %g'%entropee)
plt.xlabel('Data value')
plt.ylabel('Probability')
plt.show()

## for random variables
# create Brownian noise
N = 1123
brownnoise = np.cumsum( np.sign(np.random.randn(N)) )
fig,ax = plt.subplots(2,1,figsize=(4,6))
ax[0].plot(brownnoise)
ax[0].set_xlabel('Data index')
ax[0].set_ylabel('Data value')
ax[0].set_title('Brownian noise')
ax[1].hist(brownnoise,30)
ax[1].set_xlabel('Data value')
ax[1].set_ylabel('Counts')
plt.show()

### now compute entropy
# number of bins
nbins = 50
# bin the data and convert to probability
nPerBin,bins = np.histogram(brownnoise,nbins)
probs = nPerBin / sum(nPerBin)
# compute entropy
entro = -sum( probs*np.log2(probs+np.finfo(float).eps) )
print('Entropy = %g'%entro)
```






```
Entropy = 4.63405
```